

УНИВЕРЗИТЕТ У НИШУ
ЕЛЕКТРОНСКИ ФАКУЛТЕТ



ИМПЛЕМЕНТАЦИЈА LONG-TERM МЕМОРИЈЕ У АГЕНТСКИМ СИСТЕМИМА КОРИШЋЕЊЕМ LangMem SDK

Мастер рад
Модул: Вештачка интелигенција и машинско учење

Студент:

Јована Малиновић, бр. инд. 2133

Ментор:

Проф. др Милош Богдановић

Ниш, октобар 2026.

Универзитет у Нишу
Електронски факултет

ИМПЛЕМЕНТАЦИЈА LONG-TERM МЕМОРИЈЕ У АГЕНТСКИМ СИСТЕМИМА КОРИШЋЕЊЕМ LangMem SDK

IMPLEMENTATION OF LONG-TERM MEMORY IN AGENT SYSTEMS USING LangMem SDK

Мастер рад
Модул: Вештачка интелигенција и машинско учење

Студент: Јована Малиновић, бр. инд. 2133
Ментор: Проф. др Милош Богдановић

Задатак: *Истражити основне концепте агентских архитектура базираних на ЛЛМ-овима, ограничења контекстног прозора и потребу за екстерном меморијом, као и типове меморије релевантне за агентске системе – семантичку, епизодичку и процедуралну. Проучити механизме екстракције, консолидације и ажурирања меморијских записа током времена, стратегије претраге, као и архитектуру и функционалности LangMem SDK библиотеке у контексту интеграције са агентским оквиром LangGraph. У практичном делу, имплементирати агентски систем који користи LangMem SDK за управљање дугорочном меморијом у оквиру одабраног сценарија примене. У оквиру имплементације демонстрирати процес аутоматског издвајања релевантних информација из конверзација, њиховог чувања у перзистентном меморијском складишту, као и накнадног прибављања и коришћења у будућим интеракцијама ради персонализације одговора агента. Евалуирати систем кроз дефинисане сценарије тестирања који мере тачност прибављених меморија и конзистентност понашања агента током више сесија.*

Датум пријаве рада: 17.07.2026.
Датум предаје рада:
Датум одбране рада:

Комисија за оцену и одбрану:

1. Проф. др Милош Богдановић, Председник Комисије

2. Члан

3. Члан

ИМПЛЕМЕНТАЦИЈА LONG-TERM МЕМОРИЈЕ У АГЕНТСКИМ СИСТЕМИМА КОРИШЋЕЊЕМ LangMem SDK

САЖЕТАК

Велики језички модели у агентским системима ограничени су величином контекстног прозора и падом поузданости одговора са дужином улазног низа, па ослањање искључиво на контекст није довољно за апликације које треба да памте преференце корисника кроз више одвојених сесија. Овај рад анализира дугорочну меморију агентских система – семантичку, епизодичку и процедуралну, механизме њене екстракције, консолидације, ажурирања и претраге, као и архитектонске обрасце који је интегришу у ток извршавања агента – са посебним осврtом на *LangGraph*, оквир за изградњу агентских система заснованих на графовима, и *LangMem SDK*, библиотеку која над њим имплементира механизме дугорочне меморије. У практичном делу имплементирана је агентска апликација за претрагу и препоруку градова погодних за пресељење, у којој су над истим *LangGraph* агентом интегрисани сва три типа меморије, три режима екстракције („*hot-path*", „*background*" и „*both*"), три стратегије претраге епизода и трајно складиште засновано на *PostgreSQL/pgvector* бази. Рад је реализован у две итерације. Прототип из студијско-истраживачког рада показао је три одступања од теорије: епизодичка меморија се само уписивала, позадинска екстракција успоравала је одговор, а апликација није била отпорна на грешке алатке за ажурирање профила. Унапређени систем отклања ова одступања и евалуиран је на шест сценарија, са по пет понављања, строгом метриком која у одговору агента у новој сесији тражи само запамћене податке који се не налазе у излазу алатке за претрагу. Сва три режима са меморијом постижу строгу тачност и конзистентност кроз сесије од 0,60 до 1,00, наспрам 0,00 код агента без меморије. Када је епизодичка меморија једини активни тип меморије, агент у новој сесији у сва три режима тачно наводи разлог претходне претраге и разликује две претраге (1,00). Са оба типа меморије епизода се у овим сценаријима прибавља само у режиму „*background*"; у режимима „*hot-path*" и „*both*" агент у порукама у којима уписује профил по правилу не бележи епизоду, па прибављање не враћа ниједну епизоду, а тачан одговор (0,80, односно 1,00) потиче из поља напомена семантичког профила. Показано је и да на коришћење меморије формулација системског *prompt*-а утиче више него тренутак уписа: након њеног прецизирања разлика међу режимима екстракције знатно се смањила, али није нестала.

Кључне речи: агентски системи, велики језички модели, дугорочна меморија, *LangGraph*, *LangMem SDK*, семантичка меморија, епизодичка меморија, процедурална меморија, евалуација агентских система

IMPLEMENTATION OF LONG-TERM MEMORY IN AGENT SYSTEMS USING LangMem SDK

ABSTRACT

Large language models in agentic systems are limited by the size of their context window and by a decline in response reliability as the input sequence grows longer, so relying on context alone is not enough for applications that must remember user preferences across multiple separate sessions. This thesis analyzes long-term memory in agentic systems – semantic, episodic and procedural memory, the mechanisms for its extraction, consolidation, updating and retrieval, and the architectural patterns that integrate it into the agent's execution flow – with particular focus on *LangGraph*, a framework for building graph-based agentic systems, and *LangMem SDK*, a library that implements long-term memory mechanisms on top of it. The practical part implements an agentic application for searching and recommending towns suitable for relocation, in which all three memory types, three extraction modes („*hot-path*", „*background*" and „*both*"), three episode retrieval strategies and a persistent store based on a *PostgreSQL/pgvector* database are integrated on top of the same *LangGraph* agent. The work was carried out in two iterations. The prototype from the study research paper showed three discrepancies with the theory: episodic memory was only written and never read back, background extraction slowed down the response, and the application was not robust to errors in the profile-update tool. The improved system removes these discrepancies and was evaluated on six scenarios, with five repetitions each, using a strict metric that looks in the agent's reply in a new session only for remembered data that the search tool's output does not contain. All three memory modes reach a strict accuracy and cross-session consistency of 0.60 to 1.00, compared with 0.00 for the agent without memory. When episodic memory is the only active memory type, the agent correctly states the reason for a previous search in a new session and tells two searches apart in all three modes (1.00). With both memory types, an episode is retrieved in these scenarios only in the „*background*" mode; in the „*hot-path*" and „*both*" modes the agent usually does not log an episode in the turns in which it writes the profile, so retrieval returns no episode and the correct answer (0.80 and 1.00, respectively) comes from the notes field of the semantic profile. The results also show that the wording of the system prompt affects memory use more than the moment of writing: once the prompt was refined, the differences between the extraction modes became considerably smaller, but did not disappear.

Keywords: agentic systems, large language models, long-term memory, *LangGraph*, *LangMem SDK*, semantic memory, episodic memory, procedural memory, agent system evaluation

СПИСАК КОРИШЋЕНИХ СКРАЋЕНИЦА

СКРАЋЕНИЦЕ

ЛЛМ	Велики језички модел (Large Language Model)
АПИ	Апликациони програмски интерфејс (Application Programming Interface)
BPE	Byte Pair Encoding
MECW	Maximum Effective Context Window
MCW	Maximum Context Window
JSON	JavaScript Object Notation
RAG	Retrieval-Augmented Generation
SDK	Software Development Kit

САДРЖАЈ

1. УВОД 8

2. АГЕНТСКИ СИСТЕМИ БАЗИРАНИ НА ЛЛМ-ОВИМА 9
2.1. Основни појам и карактеристике ЛЛМ агената 9
2.2. Архитектура и основне компоненте ЛЛМ агената 10
2.3. Врсте ЛЛМ агената и обрасци понашања 12
2.4. Улога контекста у агентским структурама 13

3. КОНТЕКСТ И ОГРАНИЧЕЊА ЛЛМ МОДЕЛА 14
3.1. Контекстни прозор 14
3.2. Ограничења дужине контекста 15
3.3. Губитак информација током дугих интеракција 16
3.4. Ограничења краткорочног контекста као мотивација за екстерну меморију 17

4. МЕМОРИЈА У АГЕНТСКИМ СИСТЕМИМА 18
4.1. Појам меморије код ЛЛМ агената 18
4.2. Краткорочна и дугорочна меморија 18
4.3. Семантичка меморија 19
4.4. Епизодичка меморија 19
4.5. Процедурална меморија 20
4.6. Поређење типова меморије 20

5. УПРАВЉАЊЕ ДУГОРОЧНОМ МЕМОРИЈОМ 21
5.1. Екстракција информација из конверзација 21
5.2. Креирање и структура меморијских записа 21
5.3. Консолидација и ажурирање меморијских записа 22
5.4. Брисање и одржавање меморије 22
5.5. Претрага и коришћење меморије у генерисању одговора 23

6. АРХИТЕКТУРА СИСТЕМА СА ЕКСТЕРНОМ МЕМОРИЈОМ 24
6.1. Архитектонски обрасци меморије 24
6.2. Меморијско складиште 25
6.3. Индексирање и организација меморијских записа 25
6.4. Стратегије претраге меморије 26
6.5. RAG и меморија агента 26
6.6. Интеграција меморије са током извршавања агента 27

7. LangGraph И LangMem SDK 28
7.1. LangGraph као оквир за изградњу агентских система 28
7.2. Основни концепти и ток извршавања агента у LangGraph-у 28
7.3. Архитектура LangMem SDK-а 29
7.4. Механизми и екстракција меморије у LangMem SDK-у 29
7.5. Претрага, ажурирање и одржавање меморијских записа 30
7.6. Интеграција LangMem SDK-а са LangGraph-ом 31

8. ПРОЈЕКТОВАЊЕ И ИМПЛЕМЕНТАЦИЈА СИСТЕМА СА ДУГОРОЧНОМ МЕМОРИЈОМ 32
8.1. Дефинисање сценарија примене и захтева система 32
8.2. Архитектура и компоненте система 33
8.3. Ток информација и механизам управљања меморијом 34
8.4. Избор технологија и окружења 36
8.5. Имплементација агента и интеграција LangGraph-а и LangMem SDK-а 37
8.6. Аутоматска екстракција и перзистентно складиштење меморије 37
8.7. Структура базе података и физички облик меморијских записа 39
8.8. Претрага, прибављање и коришћење меморије у генерисању одговора 41
8.9. Ажурирање меморије током више сесија 42
8.10. Отпорност на грешке и аутоматско тестирање 42

9. ЕКСПЕРИМЕНТАЛНА ЕВАЛУАЦИЈА И АНАЛИЗА РЕЗУЛТАТА 44
9.1. Методологија евалуације и дефинисање тест сценарија 44
9.2. Почетни резултати: механизми екстракције меморије 46
9.3. Почетни резултати: типови меморије 48
9.4. Завршни резултати унапређеног система 49
9.5. Утицај формулације *prompt*-а на коришћење меморије 52
9.6. Тестирање конзистентности понашања агента кроз више сесија 54
9.7. Тестирање ажурирања меморије 55
9.8. Поштовање процедуралног правила 55

10. ЗАКЉУЧАК 56

ЛИТЕРАТУРА 57

1. УВОД

Велики језички модели (ЛЛМ) омогућили су развој агентских система способних да разумеју природни језик, резонују о задацима и самостално предузимају акције ка остваривању задатих циљева. Њихова способност да одрже конзистентну, персонализовану интеракцију кроз дуже периоде времена ипак остаје ограничена величином контекстног прозора, где сви подаци о претходним разговорима морају стати у ограничен број токена, што доводи до губитка релевантних информација чим интеракција пређе оквире једне сесије. Овај проблем постаје посебно изражен у сценаријима где се од агента очекује да „памти" кориснике и прилагођава своје одговоре на основу претходних интеракција. Као одговор на ово ограничење, све већи значај у развоју агентских система добија концепт екстерне, дугорочне меморије.

У теоријском делу рада биће представљени основни појмови везани за агентске архитектуре засноване на ЛЛМ-овима, ограничења контекстног прозора као мотивација за увођење екстерне меморије, као и типови меморије релевантни за агентске системе: семантичка, епизодичка и процедурална. Посебна пажња биће посвећена механизмима управљања дугорочном меморијом односно екстракцији, консолидацији и ажурирању меморијских записа, као и архитектури и функционалностима библиотеке *LangMem* SDK у контексту њене интеграције са агентским оквиром *LangGraph*.

У практичном делу рада имплементиран је агентски систем који помоћу *LangMem SDK*-а управља дугорочном меморијом. Систем из разговора аутоматски издваја битне информације, чува их у перзистентном складишту и касније их користи за персонализацију одговора. Евалуација се врши кроз тестне сценарије који мере тачност прибављених меморија, њихово коришћење у одговорима агента и конзистентност понашања агента током више сесија.

Рад представља надоградњу ранијег студијско-истраживачког рада у ком је развијен први прототип система. Прототип је дао добре резултате, али је показао и одступања од теорије: поједини механизми били су само делимично имплементирани, позадинска обрада успоравала је одговоре, а систем није био довољно отпоран на грешке. У овом раду та одступања су отклоњена, а систем и евалуација су проширени: унапређени су прибављање и упис меморије, обрада грешака и тестирање, а евалуација је обухватила нове сценарије и строжу меру тачности.

2. АГЕНТСКИ СИСТЕМИ БАЗИРАНИ НА ЛЛМ-ОВИМА

Појавом све способнијих великих језичких модела, дошло је до промене начина на који се ти модели користе у пракси. Уместо да само генеришу текстуалне одговоре на изоловане упите, ЛЛМ-ови се данас све чешће интегришу у сложеније системе способне да самостално планирају, користе алате и извршавају задатке који захтевају више узастопних корака [2][3]. Овакви системи, познати као ЛЛМ агенти, представљају надоградњу над традиционалном применом језичких модела, при чему ЛЛМ преузима улогу централног језгра, које усмерава ток обраде, док додатне компоненте попут модула за планирање, меморија или коришћење алата, додатно проширују његове изворне могућности [2][3]. У наставку овог поглавља биће представљени основни појам и карактеристике ЛЛМ агената, њихова архитектура и основне компоненте, преглед типичних архитектонских образаца понашања, као и улога контекста као једног од централних ограничења, која директно мотивишу потребу за дугорочном меморијом, чему је посвећен остатак овог рада.

2.1. Основни појам и карактеристике ЛЛМ агената

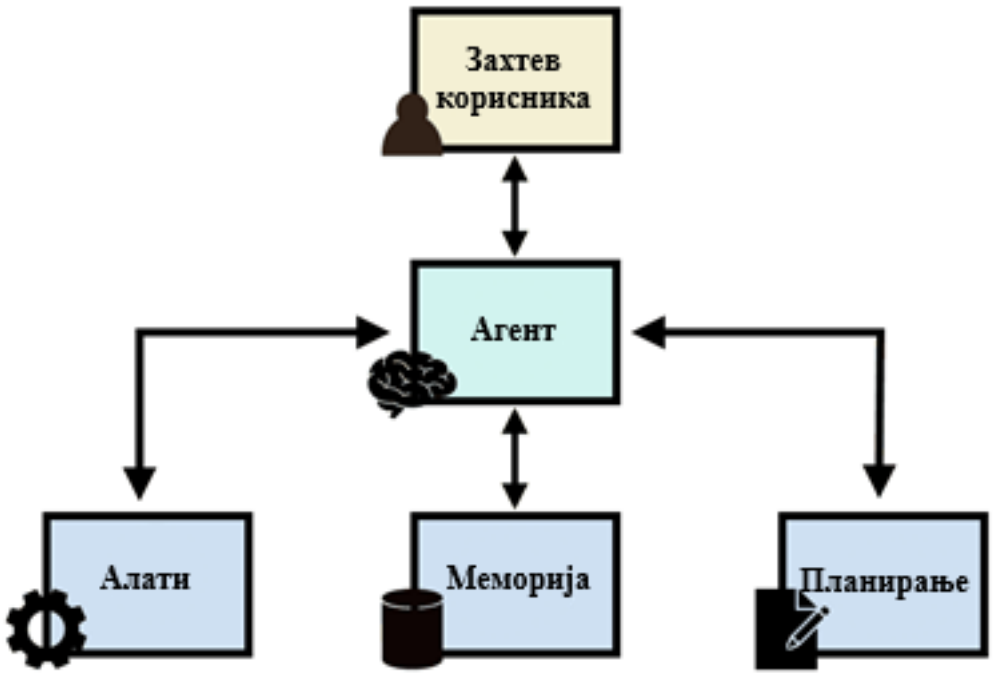
Велики језички модели у свом изворном облику функционишу као статични системи јер поседују фиксно знање стечено током тренирања и ограничени су на текстуалну интеракцију облика упит-одговор [1]. **ЛЛМ агенти** настају као одговор на ова ограничења. Реч је о системима који велики језички модел користе као централно рачунарско језгро способно да резонује о проблему, самостално саставља план решавања и, по потреби, позива спољашње алате како би задатак привео крају [2][3]. За разлику од изолованог упита на који модел даје једнократан одговор, ЛЛМ агент захтев корисника третира као задатак који се разлаже на низ мањих, међусобно повезаних корака.

Разлика између обичног упита моделу и агентског приступа најбоље се уочава на примеру. На питање попут „Колики су приходи компаније *X* у 2023. години?” модел може да одговори директно, ослањајући се на постојеће знање. Међутим, питање попут „Како се просечан дневни унос калорија у САД мењао током последње деценије и какав то може имати утицај на стопу гојазности, уз графички приказ тренда?” захтева знатно више јер агент мора да разложи проблем на подзадатке, приступи ажурним подацима преко одговарајућих алата, изврши потребне прорачуне и на крају резултат прикаже у одговарајућем облику [2]. Управо способност да се овакав сложен захтев самостално разложи, изврши корак по корак и повеже у кохерентан резултат представља суштинску карактеристику ЛЛМ агента [3].

Кључне способности ЛЛМ агената могу се сажети кроз четири категорије: планирање и вишекорачно резоновање, позивање алата и коришћење спољашњих извора, самопровера и рефлексија над сопственим одговорима, те коришћење меморије ради одржавања контекста кроз продужену интеракцију [1]. Последња наведена способност, **меморија**, представља централну тему овог рада и биће детаљно обрађена у наредним поглављима.

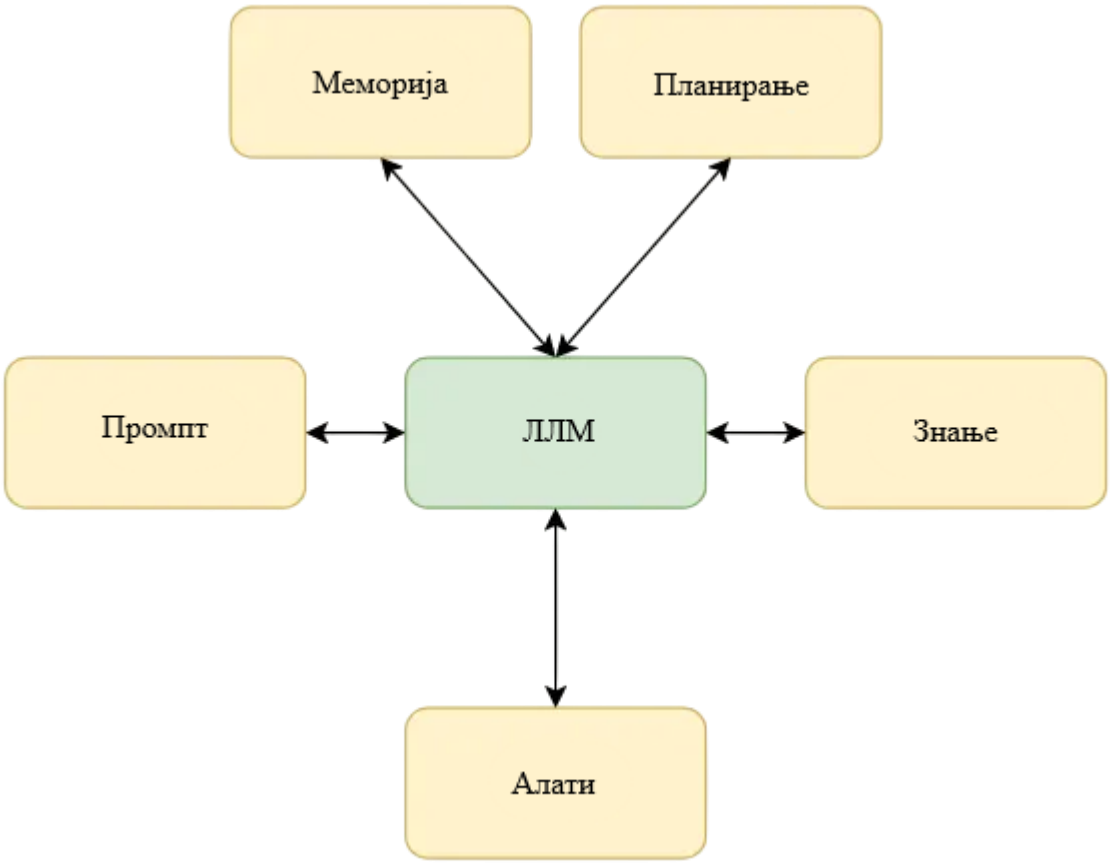
2.2. Архитектура и основне компоненте ЛЛМ агената

Архитектура ЛЛМ агената најчешће се описује кроз четири међусобно повезане компоненте које покреће захтев корисника: агент као језгро система, модул за планирање, модул за меморију и модул за коришћење алата, што је приказано на слици 2.1 [2]. Језгро чини сам велики језички модел, обучен на великим количинама текстуалних података да разуме и генерише природни језик [1][3], који у оквиру агента делује као координатор целокупног тока обраде јер тумачи захтев, одлучује које кораке треба предузети и саставља коначан одговор [2].



Слика 2.1. Основна архитектура ЛЛМ агената [2]

Иста архитектура може се посматрати и на нивоу финијих градивних елемената, где се издваја шест основних компоненти приказаних на слици 2.2: велики језички модел, *prompt*, знање, меморија, планирање и алати [3]. Прве две тичу се самог језгра. ***Prompt*** моделу даје информације о циљу, понашању и плану рада, при чему се разликује **општи *prompt***, који описује улогу, могућности и трајна ограничења агента и не мења се у зависности од конкретног задатка, од **специфичног *prompt*-а**, који носи детаље самог захтева [3]. **Знање** обухвата чињенице и информације из домена у ком агент делује, а агент до њега долази или кроз фино подешавање модела, или кроз алат који га дохвата из спољашње базе података [3]. Преостале четири компоненте одговарају модулима из претходне, крупније поделе.



Слика 2.2. Компоненте агената [3]

Модул за планирање омогућава агенту да сложени захтев разложи на низ извршивих подзадатака, уз, по потреби, механизам рефлексије којим агент критички процењује и коригује сопствени план [2][3]. Ово се постиже техникама попут **ланца мисли** (*chain-of-thought)* или **стабла мисли** (*tree-of-thoughts*), које представљају једнопутањско односно вишепутањско резоновање, док напреднији приступи попут *ReAct* и *Reflexion* оквира уводе повратну спрегу и дозвољавају агенту да коригује план на основу претходних акција и запажања из окружења [2]. Када сопствено знање модела није довољно да се задатак приведе крају, у игру улазе **алати** односно извршиве функције, АПИ-ји и спољашњи сервиси преко којих агент делује изван граница тог знања [2].

Меморија, најзад, омогућава агенту да задржи претходне мисли, акције и запажања и тако одржи континуитет резоновања кроз дужи низ корака [2]. У литератури се разликују краткорочна меморија, која одговара тренутном току мисли агента, и дугорочна меморија, која представља евиденцију интеракција које се протежу кроз недеље или месеце [3]. Комбиновањем ове две врсте агент истовремено одржава контекст тренутног задатка и знање о кориснику стечено кроз раније интеракције, при чему приступ меморији не почива искључиво на семантичкој сличности, већ и на другим факторима попут важности информације [2][3]. Могући формати у којима се меморија бележи укључују природни језик, векторска уграђивања, структуриране базе података и листе, а често се комбинују и унутар истог система [2].

2.3. Врсте ЛЛМ агената и обрасци понашања

За разлику од класичне поделе агената вештачке интелигенције по начину доношења одлука (рефлексни, циљно-оријентисани, агенти засновани на корисности), ЛЛМ агенти се у савременој литератури чешће класификују на основу начина на који комбинују аутономију, коришћење алата и сарадњу са другим агентима [4]. Према овом приступу разликују се четири основна типа.

Специфично-наменски агенти (*task-specific agents*) намењени су обављању уско дефинисаних, добро специфицираних задатака попут разврставања корисничких упита или издвајања структурираних података из докумената. Они прате унапред дефинисан ток рада, али и даље користе флексибилност ЛЛМ-а за обраду нејасних или граничних случајева, због чега се најчешће користе у продукцијским системима где су тестирање и контрола посебно важни [4].

Аутономни агенти делују уз минималну интервенцију човека и самостално одлучују на који начин ће приступити задатку. На основу широко постављеног циља, агент самостално планира процес, прикупља и анализира податке и генерише коначан резултат. Овакви агенти по правилу укључују меморију, рекурзивне петље извршавања и механизме самокорекције, а пројекти попут *AutoGPT* и *BabyAGI* представљају ране примере оваквог понашања [4].

Агенти оријентисани ка коришћењу алата (*tool-using agents*) ослањају се превасходно на спољашње алате, АПИ-је и окружења да би остварили постављени циљ. За одлучивање када, који и како користити алат ослањају се на приступе попут *ReAct* обрасца резоновања и деловања или механизма позивања функција [4].

Мултиагентни системи, уместо једног агента који обавља читав задатак, укључују више специјализованих агената који сарађују на решавању сложеног задатка. Један, на пример, прикупља податке, други их проверава, а трећи сажима резултате, при чему агенти међусобно комуницирају и размењују контекст. Оквири попут *CrewAI* и *MetaGPT* омогућавају овакву координацију [4].

Потреба за дугорочном меморијом није подједнако изражена код свих наведених типова: специфично-наменски агент може да функционише и без ње када се његов рад исцрпљује унутар једне интеракције, као код разврставања појединачних упита, док аутономни и мултиагентни системи, чије се извршавање протеже кроз дуже низове корака и више одвојених сесија, без механизма трајног памћења губе континуитет и понављају већ обављен посао.

Тип агента ипак сам по себи не одређује да ли му је дугорочна меморија потребна; одлучујуће је да ли се задатак протеже кроз више сесија и да ли одговор зависи од података о конкретном кориснику. Агент имплементиран у практичном делу рада (поглавље 8) по својој структури јесте специфично-наменски агент који користи алате: обавља уско дефинисан задатак претраге и препоруке градова, са унапред задатим скупом алатки. Његов рад се, међутим, не исцрпљује у једној интеракцији, јер корисник пресељење обично истражује кроз више одвојених посета, а свака препорука зависи од његових личних преференци, попут буџета, климе или жељене државе. Без дугорочне меморије корисник би у свакој новој сесији морао изнова да наводи исте преференце, што потврђује и понашање агента без меморије у евалуацији (поглавље 9). Управо спој уско дефинисаног задатка и персонализоване интеракције кроз више сесија чини овај сценарио погодним за испитивање дугорочне меморије: ток рада остаје контролисан и мерљив, а разлика између агента са меморијом и агента без ње директно је видљива.

2.4. Улога контекста у агентским структурама

Контекст представља укупност информација доступних моделу у тренутку генерисања одговора (претходне поруке, резултати ранијих акција агента и запажања из окружења), а обим и организација тог контекста директно одређују квалитет и кохерентност агентовог понашања [2]. Он је, међутим, коначан и ограничен величином контекстног прозора: код задатака који захтевају дуготрајно планирање, резоновање кроз дугу историју повећава ризик од грешака из којих се агент тешко опоравља, а сама граница дужине контекста директно ограничава и његову способност да се ослони на краткорочну меморију [2]. Управо зависност агента од механизама који превазилазе ово ограничење [1] представља централну мотивацију за увођење екстерне, дугорочне меморије.

3. КОНТЕКСТ И ОГРАНИЧЕЊА ЛЛМ МОДЕЛА

Претходно поглавље приказало је ЛЛМ агента као систем изграђен око великог језичког модела, проширеног модулима за планирање, меморију и коришћење алата. Сви ти модули, међутим, у крајњој линији зависе од једног заједничког, физичког ограничења самог модела односно количине текста коју модел може истовремено да „сагледа" приликом резоновања и генерисања одговора. У овом поглављу биће детаљно размотрен појам контекстног прозора, начин на који је он технички ограничен, последице које та ограничења имају на квалитет и поузданост дугих интеракција, као и то зашто управо ова ограничења представљају директну мотивацију за увођење екстерне, дугорочне меморије ЛЛМ агената.

3.1. Контекстни прозор

Контекстни прозор, често називан и дужина контекста, представља максималну количину текста, изражену у токенима, коју велики језички модел може узети у обзир у једном тренутку обраде [5][6]. Услед ауторегресивне природе генеративних модела, тренутна дужина контекста расте током самог процеса генерисања где се сваки нови излазни токен надовезује на постојећи низ и постаје део контекста на основу ког се генерише наредни токен [5]. Контекстни прозор се неформално може посматрати и као радна меморија модела јер он одређује колико дуг разговор модел може да води а да не изгуби детаље из ранијих делова интеракције, као и максималну величину документа који модел може обрадити одједном [6][7].

ЛЛМ текст не обрађује ни као низ појединачних знакова ни као низ целих речи, већ на нивоу **токена**, односно сегмената текста које пре обраде издваја **токенизатор** [5][6]. Савремени модели најчешће користе токенизацију на нивоу делова речи, попут алгоритма *Byte Pair Encoding* (*BPE*), који користе и GPT модели [5][28]. Речник таквог токенизатора гради се статистички: полази се од појединачних знакова, а затим се у корпусу за тренирање итеративно спајају парови симбола који се најчешће јављају заједно, све док речник не достигне задату величину [28]. Због тога се учестале речи, попут енглеске речи „*cat*", по правилу представљају једним токеном, док се ређе речи деле на неколико учесталијих фрагмената [6][7]. Границе тих фрагмената одређује учесталост у корпусу, а не значење нити морфолошка структура речи, па се оне могу, али не морају поклопити са префиксима, коренима и наставцима [28]. Начин на који ће тачно одређени текст бити подељен на токене зависи од метода токенизације изабраног приликом дизајнирања модела, избора величине речника, као и скупа података на ком је токенизатор трениран, због чега различити модели на различит начин токенизују исти улазни текст [5].

Величина контекстног прозора директно одређује колико информација односно корисничких порука, системског *prompt*-а, прилога и претходних одговора модела, може истовремено бити „видљиво" моделу приликом генерисања новог одговора [6]. Управо ова величина, и начин на који се она троши и попуњава током дуже интеракције, представља полазну тачку за разумевање зашто су ЛЛМ агентима потребни додатни механизми за чување и допуну информација изван граница тренутног разговора. Ти механизми, попут дугорочне меморије, преузимања релевантних информација путем RAG-а или позивања спољних алата, заједно са системским инструкцијама, историјом разговора и самим корисничким упитом чине

шири приступ познат као **контекстуално инжењерство**, чије су основне компоненте приказане на слици 3.1.

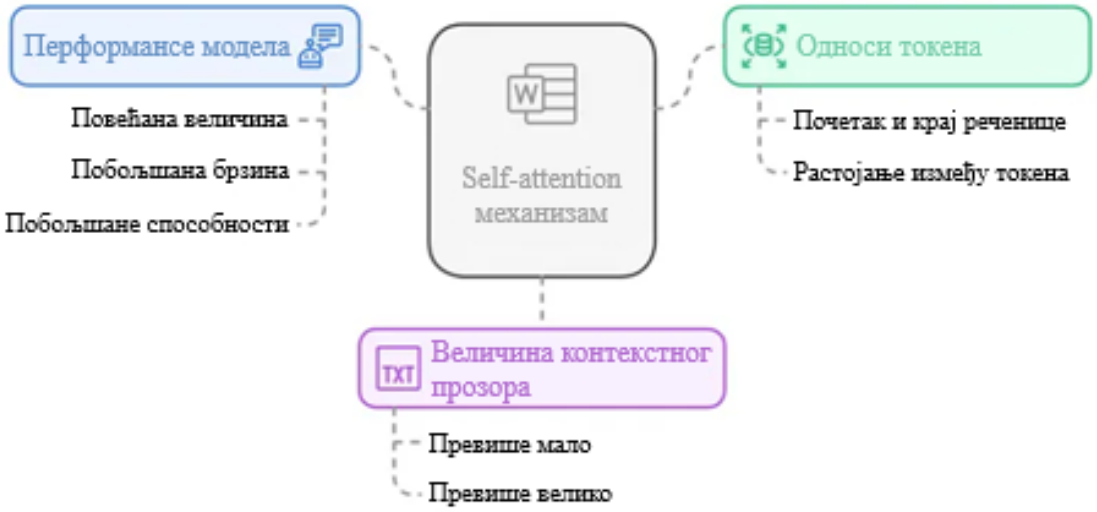


Слика 3.1. Компоненте контекстуалног инжењерства у системима заснованим на великим језичким моделима [11]

3.2. Ограничења дужине контекста

Ограничење максималне дужине контекста директно произилази из начина на који трансформерска архитектура обрађује улазни низ токена. Сваки модел има унапред одређену горњу границу броја токена које може обрадити истовремено. На пример, модел са декларисаном дужином контекста од 4.000 токена не може обрадити више од 4.000 токена, при чему сваки појединачни токен унутар тог низа пролази кроз сопствени, засебан рачунски ток обраде унутар архитектуре модела [5]. Ови токенски токови обраде нису потпуно независни. Модел се ослања на механизам ***self-attention***-а, укратко приказан на слици 3.2., који рачуна тежине које представљају колико је сваки токен у низу релевантан за остале токене у истом низу, а управо величина контекстног прозора одређује максималан број токена које модел може истовремено „узети у обзир" приликом рачунања ових тежина [6][7]. Проблем лежи у томе што рачунарски и меморијски захтеви стандардног *self-attention* механизма расту квадратно са дужином улазног низа, јер се тежине рачунају за сваки пар токена [29]. Уколико се број улазних токена удвостручи, број парова, а тиме и рачун потребан за тежине пажње, постаје четири пута већи [6][7]. Квадратно, међутим, расте само овај део модела: линеарне пројекције и *feed-forward* слојеви обрађују сваки токен засебно, па њихова цена расте линеарно са бројем токена [29]. Укупна цена модела при удвостручењу улаза зато расте мање од четири пута, али удео *self-attention* механизма у њој расте са дужином низа и

код довољно дугих низова постаје доминантан [29], што директно ограничава практичну дужину контекста коју је могуће понудити у продукцијским системима [6][7].



Слика 3.2. Механизам *self-attention*-а [7]

Ипак, декларисана максимална дужина контекста коју произвођач модела наводи не одговара нужно и стварној, практично искористивој дужини контекста. Скорашње емпиријско истраживање [8] уводи појам **максималног ефективног контекстног прозора** (MECW) односно тачке након које додатни токени више не доприносе, или чак негативно утичу на квалитет одговора модела, и показује да се овај MECW драстично разликује од **декларисаног максималног контекстног прозора** (MCW). Поједини водећи модели грешили су већ при свега 100 токена у контексту, а већина је показала озбиљан пад тачности до 1.000 токена; сви тестирани модели заостајали су за декларисаном максималном дужином контекста, и то за чак до 99% [8].

3.3. Губитак информација током дугих интеракција

Истраживање [9] показује да модели не користе подједнако ефикасно све делове доступног контекста, где је систематска анализа овог феномена спроведена кроз задатке одговарања на питања на основу више докумената и синтетичко проналажење парова кључ–вредност, контролисано мењајући позицију релевантне информације унутар контекста. Резултати показују да је тачност најбоља када се релевантна информација налази на почетку или на крају улазног низа, док значајно опада када модел мора да је пронађе и искористи негде у средини дугог контекста. Овај феномен, познат у литератури као „**изгубљен у средини**” (lost in the middle), потврђен је код више различитих модела (*GPT-3.5-Turbo, Claude, MPT-30B, LongChat-13B*), као и на синтетичком задатку проналажења, и представља једно од најчешћих ограничења дугог контекста [6][7][9].

Ефекат је толико изражен да пружање додатног контекста, у најнеповољнијем случају, може бити контрапродуктивно. Модел *GPT-3.5-Turbo* у затвореном режиму, без иједног документа у контексту, остварује тачност од 56,1% ослањајући се искључиво на знање стечено током тренирања. Када му се пружи контекст са 20

докумената међу којима се тачан одговор налази близу средине низа, тачност пада на 52,9%, испод перформанси без икаквог контекста. За поређење, када модел добије искључиво тачан документ, тачност износи чак 88,3% [9].

Осим осетљивости на позицију, дужи контекст сам по себи такође погоршава поузданост одговора, независно од тога где се релевантна информација налази. Емпиријска мерења показују да овај ефекат није занемарљив: истраживање поменуто у поглављу 3.2 бележи пад тачности код појединих водећих модела већ при 100 токена, а код већине до 1.000 токена у контексту, док стопа халуцинације код појединих модела достиже и 99% већ при свега 2.000 токена [8]. Ово практично значи да пуко проширење декларисане дужине контекстног прозора не решава проблем губитка информација већ напротив, што је контекст дужи и „запакованији" подацима, то је већа вероватноћа да ће модел пропустити или погрешно протумачити релевантну информацију негде у средини тог низа [8][9]. Овај налаз је посебно значајан за агентске системе, код којих се грешке настале губитком информације у једном кораку преносе и умножавају кроз наредне кораке ланчаног извршавања, што доводи до кумулативног пада поузданости целог система [8].

3.4. Ограничења краткорочног контекста као мотивација за екстерну меморију

Описана ограничења попут коначне величине контекстног прозора, квадратног раста цене *self-attention* механизма и губитка информација у дугим низовима, заједно указују на то да сам контекстни прозор ЛЛМ-а не може служити као једини механизам за чување информација кроз дужу интеракцију. У литератури се управо из тог разлога максимални ефективни контекстни прозор пре пореди са појмом радне (краткорочне) меморије него са дугорочном меморијом [8]. Разликовање архитектонског ограничења улазног низа од стварне функционалне способности модела постаје кључно управо у системима који симулирају дугорочну меморију или обављају резоновање кроз више сесија [8]. Другим речима, чак и када би контекстни прозор ЛЛМ-а био произвољно велик, сама природа начина на који модел троши тај простор, уз пад тачности код дужих низова и осетљивост на позицију информације, чини га непоузданим складиштем за информације које треба да преживе дужи временски период или више одвојених сесија разговора.

Из тог разлога, ЛЛМ агентима је потребан посебан, **екстерни механизам** који ће преузети улогу дугорочне меморије. Механизам који информације бира, чува, ажурира и накнадно доставља моделу само онда када су релевантне за тренутни задатак, уместо да се читава историја интеракције гура кроз ограничен и деградирајући контекстни прозор.

4. МЕМОРИЈА У АГЕНТСКИМ СИСТЕМИМА

На основу претходних поглавља показано је да контекстни прозор ЛЛМ-а не може служити као поуздано складиште информација кроз дужу интеракцију. Ово поглавље уводи појам меморије агента као механизма који то ограничење превазилази, не тако што проширује контекстни прозор, већ тако што одлучује шта од информација заслужује да буде сачувано изван њега, где ће бити чувано и када ће бити враћено назад у контекст. Биће представљен основни појам меморије код ЛЛМ агената, разлика између краткорочне и дугорочне меморије, као и три типа дугорочне меморије који ће чинити основу за остатак рада: семантичка, епизодичка и процедурална.

4.1. Појам меморије код ЛЛМ агената

Меморија агената представља механизам којим агент чува и накнадно користи информације кроз време, то јест, шта агент „памти" и на који начин [10]. За разлику од класичних ЛЛМ примена, које функционишу без стања и третирају сваку интеракцију изоловано, агент са меморијом гради континуитет односно способан је да учи из претходних интеракција, прилагођава своје понашање на основу акумулираног искуства и одржава конзистентност кроз више одвојених сесија [10][11].

Пошто ЛЛМ сам по себи нема уграђену меморију, она мора бити намерно уграђена у архитектуру система. Оно што агент у датом тренутку „зна" у потпуности је одређено садржајем његовог контекстног прозора. Дизајнирање меморије агента у суштини представља процес одлучивања који токени улазе у контекстни прозор и на који начин су организовани [10][11]. Интеграција меморије агенту омогућава дубљу персонализацију, континуитет кроз више сесија, већу ефикасност избегавањем понављања истих питања, и способност сложенијег резоновања ослањањем на релевантне чињенице и искуства из прошлости [11].

4.2. Краткорочна и дугорочна меморија

Меморија агента конвенционално се дели на краткорочну и дугорочну, слично подели која се користи и код људске когниције [11]. Критеријум поделе није то где се информација налази у тренутку позива модела, већ где се трајно чува и колико дуго траје. **Краткорочна** (радна) **меморија** обухвата информације које постоје само у контекстном прозору током текуће сесије: системска упутства, недавну историју разговора, тренутни задатак и дефиниције алата. Брза је и неопходна за тренутни задатак, али привремена, ограничене величине и мора бити реконструисана при сваком позиву модела [10][11].

Дугорочна меморија, насупрот томе, чува се у екстерном складишту података, најчешће у векторској бази или сличном перзистентном механизму, и омогућава агенту да чува и накнадно призива информације кроз више одвојених сесија. Управо у дугорочној меморији се одвија права персонализација и „учење" агента. Када је релевантна за тренутни задатак, она се учитава у контекстни прозор и тада привремено постаје део радне меморије, али њен трајни запис остаје у складишту, где се може ажурирати независно од тока разговора [11]. Из истог разлога поједини системи у

дугорочну меморију сврставају и уређиве блокове меморије који су стално прикачени уз контекст: иако их модел види при сваком позиву, они се чувају и мењају изван контекста и преживљавају крај сесије, док се као краткорочна меморија третирају само поруке у активном бафери разговора [10]. На исти начин се у систему имплементираном у овом раду профил корисника уграђује у системски *prompt* при сваком позиву модела, а трајно чува у складишту (поглавље 8.8). Дугорочна меморија даље се дели на семантичку, епизодичку и процедуралну [10].

4.3. Семантичка меморија

Семантичка меморија чува структурисано чињенично знање и концептуално разумевање попут правила, дефиниција и односа о којима агент резонује [11][12]. За разлику од епизодичке меморије, која чува конкретна лична искуства, семантичка меморија представља опште знање које није везано за конкретан случај. На пример, да корисник преферира *Python* у односу на *JavaScript*, или, код правног асистента, да се уговорно право разликује од кривичног [11][12]. Семантичка меморија одговара на питање „шта" – шта агент зна о свету и о кориснику [11].

У пракси, семантичка меморија најчешће се моделује кроз структуриране базе знања или графове знања где ентитети и њихови односи могу бити упитани, али многи агенти чувају и неструктурирано знање из домена у векторским базама и приступају му путем RAG механизама [12]. Временом, обрасци издвојени из епизодичке меморије могу бити пречишћени у семантичко знање, омогућавајући агенту да генерализује изван појединачних искустава [12].

4.4. Епизодичка меморија

Епизодичка меморија омогућава агенту да се сети конкретних догађаја и искустава из своје оперативне историје – шта се десило, када се десило и какав је био исход [11][12]. На пример, финансијски саветник заснован на ЛЛМ-у уз епизодичку меморију не располаже само општим принципима инвестирања већ он памти да је пре три месеца препоручио одређени портфолио кориснику А, и да та препорука није дала добре резултате [12]. Оваква конкретна искуства обликују будуће препоруке на начин на који опште знање не може.

Епизодичка меморија најчешће се имплементира коришћењем векторских база података које омогућавају семантичко претраживање кроз прошле епизоде где уместо тачног подударања, агент може пронаћи искуства која су концептуално слична тренутној ситуацији, чак и када се детаљи разликују [12]. У пракси, епизодичка меморија чува структурисане записе интеракција: временске ознаке, идентификаторе корисника, предузете акције и запажене исходе, који потом служе агенту приликом доношења одлука, омогућавајући облик резоновања заснованог на претходним случајевима [12].

4.5. Процедурална меморија

Процедурална меморија представља научене вештине и обрасце понашања које агент може да изврши аутоматски, без потребе да поново „размишља” о сваком кораку – одговара на питање „како” агент обавља одређени задатак [11][12]. Слично начину на који особа временом научи да куца или вози са све мање свесног напора, процедурална меморија агента омогућава да се често коришћене процедуре претворе у поуздане, кеширане рутине. Агент за корисничку подршку који по стоти пут обрађује захтев за ресетовање лозинке не мора поново да резонује о читавом току посла јер секвенца корака постаје устаљена рутина [12].

Процедурална меморија не елиминише резоновање у потпуности, већ смањује потребу за понављаним промишљањем, ослобађајући капацитет за нове ситуације [12]. Овај тип меморије може настати кроз учење поткрепљивањем, фино подешавање модела, или експлицитно дефинисане токове рада [11][12]. На пример, ако више корисника агенту каже да су његови сажеци предугачки, то запажање се временом може претворити у трајно правило понашања [11][12].

4.6. Поређење типова меморије

Три типа дугорочне меморије нису међусобно искључиви већ их најспособнији системи комбинују. Епизодичка меморија сама учинила би агента превише персонализованим без општег знања; семантичка меморија сама учинила би га упућеним, али неспособним да учи из искуства; процедурална меморија сама учинила би га добрим у извршавању програмираних задатака, али неприлагодљивим пред новим ситуацијама [12]. Разлика међу њима најјасније се уочава кроз питање на које свака одговара и начин на који се типично имплементира, што је сажето у табели 4.1.

Тип	Питање	Шта чува	Типична имплементација
Семантичка	„шта”	чињенице, преференце, знање о домену	структуриране базе, графови знања, векторске базе (RAG)
Епизодичка	„шта се десило и када”	конкретни догађаји, акције и исходи	векторске базе са временским ознакама
Процедурална	„како”	правила и обрасци понашања	системски *prompt*, фино подешавање, токови рада

Табела 4.1. Поређење типова дугорочне меморије [11][12]

Ова разлика није само теоријска јер одређује и који тип меморије треба нагласити у конкретној примени, у зависности од тога да ли је агенту првенствено потребно да памти корисника, да познаје домен у ком делује, или да поуздано извршава устаљене процесе [12]. Зато избор архитектуре меморије за агентски систем у пракси представља питање тога које од ова три питања агент најчешће мора себи да постави.

5. УПРАВЉАЊЕ ДУГОРОЧНОМ МЕМОРИЈОМ

Док се претходно поглавље бавило питањем шта агент памти, ово поглавље усмерено је на питање како се то памћење остварује у пракси. Управљање дугорочном меморијом обухвата читав животни циклус једног меморијског записа, од тренутка у којем се из разговора издваја релевантна информација, преко начина на који се она структурира и чува, па све до каснијег ажурирања, застаревања и, најзад, употребе приликом генерисања одговора.

5.1. Екстракција информација из конверзација

Сирове конверзације су „хаотичне" јер у типичним логовима агентских интеракција, значајан део токена чине уобичајена конверзација, понављања или пролазно резоновање које не носи трајну вредност [14]. **Екстракција меморије** представља процес у ком велики језички модел анализира садржај разговора и издваја само оне делове који заслужују да буду трајно сачувани. Типичан образац изгледа овако: агент „прочита" реченицу корисника (нпр. „Преферирам *Python*, не *JavaScript*"), а модел за екстракцију из ње издваја структурирану чињеницу заједно са пратећим метаподацима, идентификатором корисника, временском ознаком и, по потреби, назнаком да нова информација негира претходно сачувану [14].

Овај процес обично се остварује кроз компоненту која се назива **менаџер меморије**. Она прима конверзацију и тренутно стање меморије, а затим упитом ка ЛЛМ-у одлучује како то стање треба проширити или консолидовати [13]. Кључна одлука приликом дизајнирања екстракције тиче се обима: ако систем прекомерно издваја и чува превише детаља, то умањује прецизност претраге а ако недовољно издваја, долази до губитка релевантних података [13]. Због тога проналажење равнотеже између ова два ризика представља једно од централних питања приликом пројектовања система.

5.2. Креирање и структура меморијских записа

Меморијски запис ретко се чува као сиров текст односно структурира се на начин који олакшава накнадно претраживање, најчешће у формату попут JSON-а или сличне шеме са јасно дефинисаним пољима [17]. У пракси се издвајају два основна начина организовања семантичке меморије. Први приступ чува меморије као **засебне, независне записе** где свака нова информација постаје нов запис, чиме се прати знање кроз много интеракција без губитка детаља. Други приступ одржава један **обједињени** документ који представља тренутно стање целог знања о кориснику или задатку где нова информација не додаје нов запис, него ажурира постојећи документ [17]. Избор зависи од намене: обједињени документ погодан је када је важно само тренутно стање и лак преглед, док одвојени записи омогућавају праћење детаљне историје без губитка информација. Добро дизајниран систем меморије често комбинује оба приступа односно чува и запис кључних претходних интеракција и профил онога што је агент временом научио [17].

Епизодичка меморија обично се структурира другачије, не као изолована чињеница, него као целовит запис ситуације: контекст у ком се нешто десило, предузета акција и остварени исход, чиме агенту касније служи као пример на основу ког може закључити на сличан начин у сродној ситуацији. Процедурална меморија, најзад, ретко се чува као засебан запис у бази података. Чешће се уграђује директно у упутства која усмеравају понашање агента, у виду правила која се временом усавршавају на основу повратних информација.

5.3. Консолидација и ажурирање меморијских записа

Појам **консолидације** потиче из когнитивне психологије, где означава процес којим мозак краткорочна сећања преводи у трајнији облик складиштења [16]. У контексту ЛЛМ агената, консолидација означава интеграцију нове информације са већ постојећим меморијским записима: уместо да сваку нову чињеницу дода као изолован запис, систем процењује да ли она допуњује, потврђује или противречи нечему што је већ сачувано.

У пракси, консолидација периодично прегледава постојеће складиште. Записи чија је семантичка сличност изнад одређеног прага постају кандидати за спајање, при чему ЛЛМ одлучује како да разреши евентуалан сукоб, након чега следи уклањање дупликата, док се показатељи релевантности ажурирају на основу учесталости коришћења [14]. Овакав приступ смањује простор потребан за складиштење и повећава прецизност претраге у поређењу са приступом који сирове податке само гомила [14]. У литератури се овај процес назива и **обогаћивањем меморије**, а подразумева успостављање равнотеже између стварања нових записа и консолидовања постојећих, уз могућност да програмер додатно подеси у ком правцу ће та равнотежа бити нагнута [13].

Посебан случај консолидације настаје када нова информација не допуњује постојећи запис, него мења његову вредност. За такве ситуације у пракси постоје два приступа. **Потпуно поновно писање** подразумева да систем, на основу нове интеракције, изнова генерише читав меморијски запис – једноставније је, али ризикује губитак детаља који нису директно поменути у последњој интеракцији. **Ажурирање на нивоу појединачних поља** подразумева да систем препозна тачно која су поља промењена и измени само њих, задржавајући остатак записа непромењеним – прецизније је, али захтева пажљивије осмишљено упутство за модел [17].

Најзахтевнији облик овог проблема јавља се када нова информација директно противречи старој. Контекстни прозор сам по себи ту не помаже, јер пасивно чува и противречне уносе, без механизма за поравнање или временско означавање, што доводи до губитка тачности код задатака који зависе од временског редоследа информација [14]. Управо способност да меморија буде ажурирана, а не само гомилана, представља једну од кључних предности намерно пројектованог система дугорочне меморије у односу на пуко проширивање контекстног прозора.

5.4. Брисање и одржавање меморије

Одржавање меморије подразумева и супротан процес од чувања, то јест, одлучивање шта треба уклонити. Агент који памти апсолутно све временом престаје да

памти било шта корисно: акумулација небитних или застарелих записа доводи до **оптерећења меморије**, што чини претрагу споријом, скупљом и мање прецизном. Дизајнирање стратегије брисања захтева пажљиво одмеравање јер агресивно брисање ризикује да уклони ретке, али важне детаље, док прекомерно чување води ка проблему оптерећења.

У пракси, брисање се ослања на комбинацију сигнала: старости записа, учесталости и скорашњости коришћења, као и експлицитних сигнала противречности [14]. Неки системи (нпр. *AgeMem*) овај процес третирају као експлицитну алатку коју агент сам позива по потреби: поред алатки за додавање и ажурирање дугорочне меморије, агенту је на располагању и алатка за брисање записа. Њена употреба учи се кроз повратну спрегу током тренирања, а не кроз унапред задату хеуристику [15]. Ово показује да брисање не мора бити пасивна функција одржавања базе, него активна одлука коју агент доноси на основу процене трајне вредности информације.

5.5. Претрага и коришћење меморије у генерисању одговора

Претрага меморије често се поједностављено изједначава са проналажењем записа који су семантички најближи тренутном упиту, али релевантност у пракси зависи од више фактора истовремено. Поред семантичке сличности, на њу утичу и „**важност**” саме меморије и њена „**снага**”, то јест функција тога колико скоро и колико често је одређени запис коришћен у прошлости [13]. Записи који су често призивани временом добијају на тежини, док они који дуго нису били потребни постепено губе предност, чиме претрага постаје осетљива не само на садржај упита, него и на историју коришћења самог складишта. Конкретне технике којима се ово остварује предмет су наредног поглавља.

Пронађени записи затим се враћају назад у контекстни прозор, у тренутку када су релевантни за задатак који агент обавља. Овај корак затвара животни циклус: информација издвојена и сачувана у ранијој интеракцији поново постаје део краткорочне, радне меморије модела, али сада обогаћена контекстом целе историје корисника.

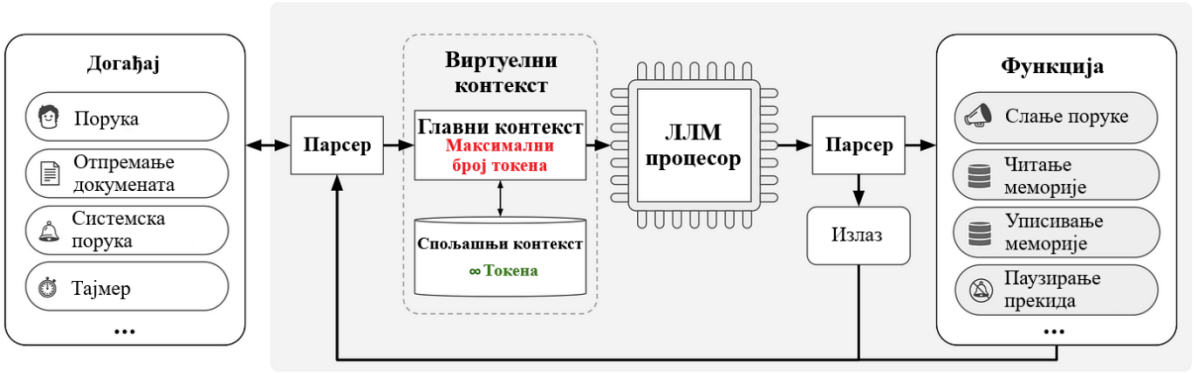
Из практичних разлога, у контекст се не убацује читав садржај меморијског складишта, него само уско издвојен, најрелевантнији подскуп [14]. У продукцијским системима то је типично неколико записа ограничених на свега неколико стотина токена, како би се избегло понављање управо проблема попут пада тачности у дугим низовима и осетљивости модела на позицију информације унутар контекста [9][14]. На овај начин коришћење меморије не представља пуко „додавање још текста” контексту, него намерно, селективно допуњавање тренутног упита информацијама које модел самостално не би могао да се сети без екстерног складишта.

6. АРХИТЕКТУРА СИСТЕМА СА ЕКСТЕРНОМ МЕМОРИЈОМ

Претходна поглавља описала су шта агент памти и кроз какав животни циклус пролази појединачни меморијски запис. Ово поглавље спаја те елементе у целовиту архитектуру система: кроз какве се архитектонске обрасце краткорочна и дугорочна меморија организују у једну функционалну целину, где се дугорочни записи физички чувају, како се индексирају и претражују, у каквом је односу овакав систем са добро познатим приступом претраживањем потпомогнуте генерације (RAG) и, на крају, како се меморија уграђује у сам ток извршавања агента.

6.1. Архитектонски обрасци меморије

У пракси, системи агентске меморије групишу се у неколико препознатљивих архитектонских образаца. **Најједноставнији образац** чува целокупну меморију унутар самог *prompt*-а, без икакве спољашње инфраструктуре, потпуно јасан, али строго ограничен капацитетом контекстног прозора и склон постепеном губитку детаља кроз узастопна сажимања. **Знатно распрострањенији образац** у продукцијским системима комбинује радну меморију унутар контекстног прозора са екстерним складиштем за дугорочне записе, при чему механизам претраге у сваком кораку убацује релевантне записе назад у контекст. Овај образац данас представља окосницу већине продукцијских агентских система, а главни инжењерски изазов лежи у квалитету саме претраге, а не у сложености инфраструктуре [19]. Трећи, **најсложенији образац** уводи више слојева меморије којима управља научена или подстакнута контролна логика, при чему сам агент одлучује када и како да премешта информације између слојева [19].



Слика 6.1. Хијерархијска архитектура меморије инспирисана организацијом рачунарских система [18]

Овај трећи образац илуструје архитектура инспирисана организацијом рачунарских система са слике 6.1.: примарни контекст третира се као брза, али ограничена радна меморија, подељена на статични системски *prompt*, динамички радни простор за резоновање и *First-In, First-Out* бафер недавних порука, док спољашњи, практично неограничен слој обухвата претраживу базу целокупне историје интеракција и архивско, векторски индексирано складиште за дугорочно знање [18]. У оквиру шире класификације архитектонских образаца меморије агента, иста архитектура среће се и

под другачијом терминологијом – hot/warm/cold меморија уместо RAM/recall/archival, при чему се увек наглашава иста кључна одлика: агент сам активно управља сопственом меморијом преко позива функција, одлучујући шта ће задржати, сажети или архивирати, уместо да пасивно прима убачен контекст [20]. Проток информација између ова два нивоа регулисан је **механизмом преношења**: када употреба токена у примарном контексту достигне одређени праг, систем упозорава агента, који затим самостално процењује шта је од тренутног садржаја најмање критично, сажима га и премешта у одговарајући спољашњи ниво [18].

6.2. Меморијско складиште

Начин на који је дугорочна меморија физички представљена директно одређује на које врсте упита систем може ефикасно да одговори. **Векторски индексирана складишта** представљају сваки меморијски запис као густи ембединг вектор и омогућавају приближну претрагу најближих суседа. Добро скалирају на милионе записа, али губе експлицитне односе међу записима и могу одговорити на питање „шта личи на ово", али не и на питање „шта је узроковало ово" [19].

Структурирана складишта – релационе базе, кључ-вредност мапе и графови знања, чувају релационе односе и подржавају сложеније упите, по цену унапред дефинисане шеме [19]. **Графовске базе** меморију представљају као чворове и везе између ентитета, што их чини погодним за упите који захтевају праћење односа између више повезаних чињеница [14]. **Извршни репозиторијуми**, најзад, чувају меморију у облику извршивог кода или дефинисаних процедура које агент може директно да позове, заобилазећи потребу за поновним генерисањем [19].

У продукцији су **хибридна складишта** постала уобичајена: векторска претрага служи за брзо проналажење семантички сличних кандидата, а графовска структура за проверу односа између ентитета, при чему се коначна оцена релевантности рачуна као пондерисана комбинација оба сигнала [14]. Избор складишта у пракси представља компромис јер векторска складишта пружају брзину и семантичку флексибилност, графовска релациону прецизност, а хибридни приступи покушавају да задрже предности оба, по цену веће сложености система [14].

6.3. Индексирање и организација меморијских записа

Начин на који се меморијски записи индексирају директно утиче на прецизност накнадне претраге. **Ситнозрнато индексирање**, на нивоу појединачних чињеница или реченица, омогућава прецизно проналажење, али ризикује да разбије вишекорачно резоновање на бесмислене фрагменте; **крупнозрнато индексирање**, на нивоу читавих сесија или дужих пасуса, чува контекст, али пригушује сигнал шумом [19]. Као решење се у пракси намеће индексирање на више нивоа гранулације истовремено, при чему систем за претрагу адаптивно бира одговарајућу резолуцију записа у зависности од упита [19]. Технички, ово се најчешће остварује комбиновањем густе, уграђивачке претраге са ретком, кључно-заснованом претрагом и филтрирањем по метаподацима попут временске ознаке, врсте алата или ознаке задатка [19].

Поред гранулације, на проналажење утиче и логичка организација складишта. Меморије се често групишу у **хијерархијске просторе имена**, на пример по

организацији, кориснику и апликацији унутар које агент делује, чиме се обезбеђује изолација података и олакшава накнадно проналажење записа релевантних за конкретан контекст рада [13]. Оваква сегментација важна је и из практичних разлога изван саме претраге, јер омогућава да се меморија једног корисника обрише или измени независно од осталих.

6.4. Стратегије претраге меморије

На квалитет претраге не утиче само начин на који су записи индексирани, него и то како је формулисан сам упит. Тренутни унос корисника често је лош упит за претрагу: питање попут „Зашто је то отказало?" захтева да систем пронађе конкретан извештај о грешци из раније сесије, а не реченицу која му је семантички најсличнија. Стратегије решавања овог проблема укључују **преформулисање упита** помоћу ЛЛМ-а пре саме претраге, слање више варијанти упита паралелно уз накнадно спајање резултата, и коришћење тренутног подциља задатка као додатног сигнала [19]. Поједини системи иду корак даље и уче модел да сам процени да ли је претрага уопште потребна за дати упит, чиме се смањује непотребно кашњење код упита који не захтевају приступ меморији [19].

Поред претраге засноване на сличности, системи по правилу нуде и директан приступ конкретној меморији по кључу, као и филтрирање по метаподацима, што омогућава прецизнију претрагу када је унапред познато шта се тачно тражи [13]. У пракси се ретко ослања на једну стратегију већ се комбинују **кључна претрага**, **семантичка претрага** и **филтрирање по метаподацима**, а у сложенијим системима и **хибридна претрага** која обједињује више сигнала истовремено.

6.5. RAG и меморија агента

Retrieval-Augmented Generation (RAG) представља технику проширивања одговора великог језичког модела резултатима претраге над спољашњом базом докумената, и историјски је био један од првих начина да се ЛЛМ повеже са знањем изван граница сопственог тренинга. Меморија агента и RAG деле заједничку техничку основу јер се оба ослањају на индексирање и претрагу спољашњих података али служе различитој сврси. RAG типично претражује статичну, унапред припремљену базу докумената ради одговарања на чињенична питања, док меморија агента чува динамичке, временски означене записе о самом кориснику и историји интеракције, које систем сам генерише, ажурира и, по потреби, брише [19].

Проширивање контекстног прозора само по себи не чини RAG нити меморију сувишним јер ниједан од ова два механизма не пружа оно што дужи контекст сам по себи не може: перзистентно чување кроз сесије, структурирану организацију знања, селективно проналажење кроз историју, или механизме управљања попут брисања и контроле приступа [19]. У пракси, добро пројектован агентски систем третира RAG као једну од могућих стратегија претраге унутар шире архитектуре дугорочне меморије, а не као њену замену.

6.6. Интеграција меморије са током извршавања агента

Механизми описани у претходном поглављу не извршавају се сви у истом тренутку рада агента, него се распоређују на два комплементарна пута: пут писања и пут читања. **Пут писања** обухвата одлучивање о томе шта од тренутне интеракције заслужује да буде сачувано, заједно са припремом записа за складиштење: одбацивањем небитних детаља, нормализацијом датума, имена и количина, уклањањем дупликата и означавањем метаподацима [19]. Оптималан праг филтрирања при том није универзалан, него зависи од домена примене: систем у медицинском домену не сме да пропусти помен алергије, док систем за корисничку подршку мора да задржи уговорне обавезе, уз толерантнији праг за неформалне детаље [19].

Кључна инжењерска одлука на путу писања тиче се тренутка извршавања. Уколико се екстракција одвија унутар самог корака одговора, меморија је ажурна одмах, али корисник плаћа додатно кашњење; уколико се извршава као засебан процес након што је одговор већ послат, одговор остаје брз, али нова информација постаје доступна тек са одлагањем. Избор између ова два распореда, или њихова комбинација, зависи од тога да ли примена захтева тренутно ажурну меморију или толерише кашњење у замену за брзину.

Пут читања, са друге стране, одлучује када и колико информације из меморије треба укључити у тренутни корак резоновања. Уобичајене оптимизације укључују двостепену претрагу, у којој брз и груб филтер претходи прецизнијем поновном рангирању, процену тога да ли је претрага уопште потребна за дати корак, динамичко распоређивање расположивог простора у контексту између меморије и тренутног задатка, и кеширање записа који се често користе [19]. Свака од ових оптимизација представља компромис између тачности, кашњења и цене извршавања.

Посматрана заједно, ова два пута чине меморију активним учесником у сваком кораку агентовог рада, а не пасивним архивом који се повремено консултује. Управо на овом месту теоријски опис прелази у питање конкретне реализације: како се описани обрасци, складишта и путеви писања и читања остварују кроз постојеће технолошке алате?

7. LangGraph И LangMem SDK

Претходна поглавља описала су меморију агента на општем, технолошки-неутралном нивоу односно типове меморије, механизме управљања и архитектонске обрасце који их повезују у целину. Ово поглавље спушта ту теорију на ниво конкретне технологије коришћене у практичном делу овог рада: *LangGraph*, оквир за изградњу агентских система заснованих на графовима, и *LangMem SDK*, библиотеку изграђену над *LangGraph*-ом која имплементира управо оне механизме дугорочне меморије описане у претходним поглављима.

7.1. LangGraph као оквир за изградњу агентских система

LangGraph је оквир ниског нивоа за оркестрацију агентских система, намењен изградњи дуготрајних, стабилних агената и токова рада [21]. За разлику од виших нивоа апстракције, *LangGraph* не апстрахује *prompt*-ове нити архитектуру агента већ представља агентски систем експлицитно као граф, где сваки корак обраде чини чвор, а везе између корака чине гране [21]. Оваква ниска апстракција пружа фину контролу над понашањем агента: могуће је комбиновати детерминистичке, унапред дефинисане кораке логике са корацима у којима одлуку доноси сам ЛЛМ, унутар истог графа [21].

Кључне предности *LangGraph*-а обухватају подршку за циклусе у извршавању, уграђену перзистенцију стања, механизме за укључивање човека у петљу одлучивања и, за овај рад најбитније, свеобухватну подршку за меморију, то јест, истовремено краткорочну радну меморију унутар једне сесије и дугорочну меморију кроз више сесија [21]. *LangGraph* је развијен као засебан пакет компаније *LangChain*, инспирисан системима *Pregel* и *Apache Beam*, али се може користити независно од остатка *LangChain* екосистема [21].

7.2. Основни концепти и ток извршавања агента у LangGraph-у

Средишњи концепт *LangGraph*-а јесте **стање** – дељена структура података која представља тренутни снимак апликације у датом тренутку извршавања [26]. Стање се преноси дуж грана графа између чворова, при чему сваки чвор прима тренутно стање као улаз и враћа делимично ажурирање тог стања. **Чвор** представља појединачну функцију која извршава један корак обраде, док **гране** одређују редослед извршавања: могу бити безусловне или условне, усмеравајући ток у зависности од садржаја тренутног стања [26].

Ови елементи повезују се у граф помоћу класе ***StateGraph***, дефинисане унапред задатом шемом стања, уз обавезно дефинисану улазну тачку и услов завршетка [26]. За разлику од линеарних ланаца обраде, *LangGraph* дозвољава и цикличне графове – граф се може вратити на претходно посећени чвор, што агентским системима омогућава да понављају кораке резоновања, позивају алате више пута или исправљају сопствене грешке пре него што дају коначан одговор [21].

Оно што чини стање *LangGraph*-а посебно значајним за тему овог рада јесте разлика коју *LangGraph* успоставља између два комплементарна механизма перзистенције. **Контролне тачке** чувају стање нити кроз низ контролних тачака и

намењене су краткорочној меморији унутар нити: наставку разговора, механизмима укључивања човека у петљу, враћању уназад кроз историју извршавања и отпорности на грешке [22]. **Складишта**, насупрот томе, чувају податке дефинисане апликацијом изван самог стања графа и намењена су дугорочној меморији између нити: корисничким преференцама, чињеницама и знању дељеном кроз више одвојених нити разговора [22].

Ова подела директно одговара оном што је раније описано као краткорочна наспрам дугорочне меморије, сада реализовано кроз конкретне технолошке компоненте: контролна тачка чува стање сваке нити при сваком кораку извршавања графа, док складиште преживљава изван граница појединачне нити и доступно је свим нитима које деле исти простор имена [22]. У већини реалних примена агент користи оба механизма истовремено.

7.3. Архитектура LangMem SDK-а

LangMem је *open-source Python SDK* изграђен над *LangGraph*-ом, намењен агентима да уче и прилагођавају се на основу сопствених интеракција кроз време [23]. Пружа алате за издвајање важних информација из конверзација, оптимизацију понашања агента кроз рафинисање *prompt*-ова и одржавање дугорочне меморије, чиме агенту омогућава да континуирано унапређује своје одговоре, персонализује их и одржи конзистентно понашање кроз више сесија [23]. Архитектура *LangMem*-а почива на четири кључне компоненте [13][23]:

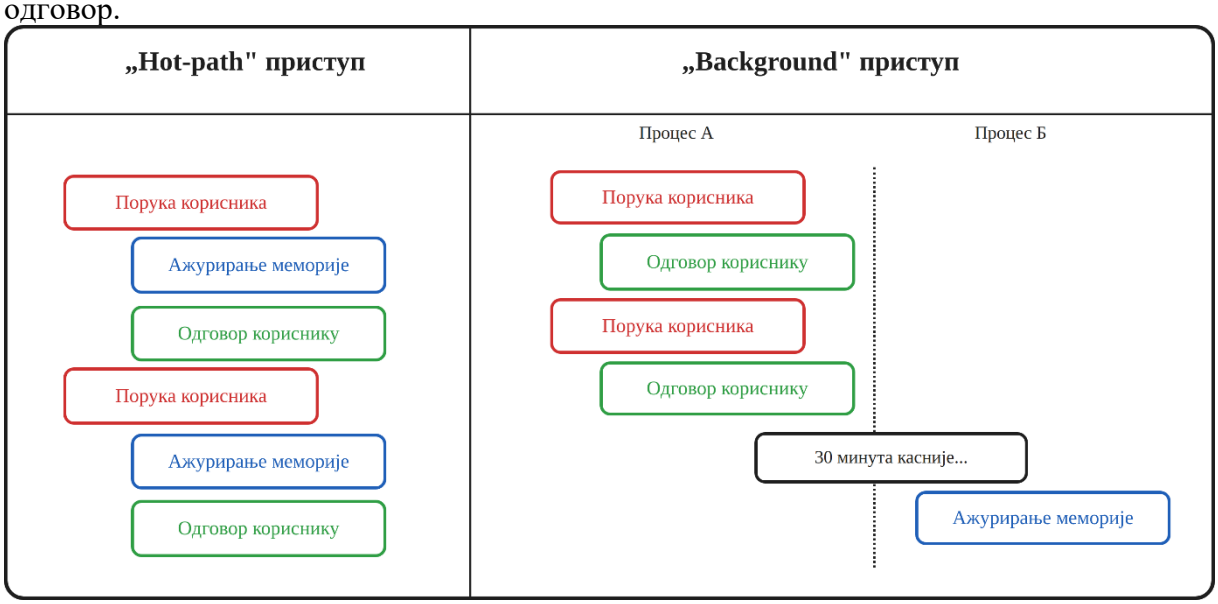
- **основни меморијски АПИ**, независан од конкретног складишта података;
- **алатке за управљање меморијом** које агент може позвати током активне конверзације, „у *hot-path*-у";
- **позадински менаџер меморије**, који аутоматски издваја, консолидује и ажурира знање агента и
- **нативна интеграција** са *LangGraph*-овим складиштем за дугорочну меморију, доступна подразумевано у свим *LangGraph Platform* окружењима [23].

7.4. Механизми и екстракција меморије у LangMem SDK-у

LangMem подржава два комплементарна начина формирања меморије, директно одговарајући подели описаној раније. Приступ „*hot-path*" подразумева да агент сам, током саме интеракције, активно одлучује шта треба запамтити, позивајући наменске алатке за управљање меморијом. Приступ „*background*" подразумева да посебан процес, независан од главног тока разговора, накнадно анализира конверзацију и аутоматски издваја релевантне меморије, без директног учешћа агента у тренутку разговора [23][24][25].

Свака стратегија носи различите компромисе: „*hot-path*" приступ даје тренутна, транспарентна ажурирања меморије, али уноси додатно кашњење у одговор агента, како је приказано на слици 7.1. „*Background*" приступ не успорава одговор, али уводи кашњење у само ажурирање меморије где нова информација неће бити одмах доступна, него тек након што позадински процес заврши обраду [24][25], како је приказано на слици 7.1. Избор између њих, или њихова комбинација, зависи од тога да ли апликација захтева ажурну меморију у истом тренутку, или толерише кашњење у замену за бржи

одговор.



Слика 7.1. Однос „*hot-path*” и „*background*” приступа у чувању меморије у *LangMem SDK*-у [13]

У ***„hot-path”*** приступу, екстракција се остварује позивањем наменских алатки директно унутар агентовог тока резоновања где алатка ***create_manage_memory_tool*** омогућава агенту да сам креира, ажурира или брише меморијске записе, чувајући их у именованом простору унутар складишта [24]. Агент сам одлучује шта и када треба запамтити; довољно је да корисник у разговору помене релевантну информацију, и агент ће самостално позвати алатку за чување [23].

У ***„background”*** приступу, екстракција се остварује помоћу компоненте направљене преко функције ***create_memory_store_manager***, која прима историју разговора и самостално издваја меморије из ње. За разлику од „*hot-path*” приступа, агент овде не мора експлицитно да позове алатку већ је довољно да се позадински менаџер меморије позове након генерисања одговора, чиме се процес издвајања потпуно одваја од главног тока разговора [25].

7.5. Претрага, ажурирање и одржавање меморијских записа

Симетрично алатки за управљање меморијом, *LangMem* нуди и алатку ***create_search_memory_tool***, која агенту омогућава да експлицитно претражи претходно сачуване меморије унутар истог простора имена [23]. Када корисник постави питање везано за претходну интеракцију, агент може самостално позвати ову алатку да пронађе релевантне записе пре него што генерише одговор.

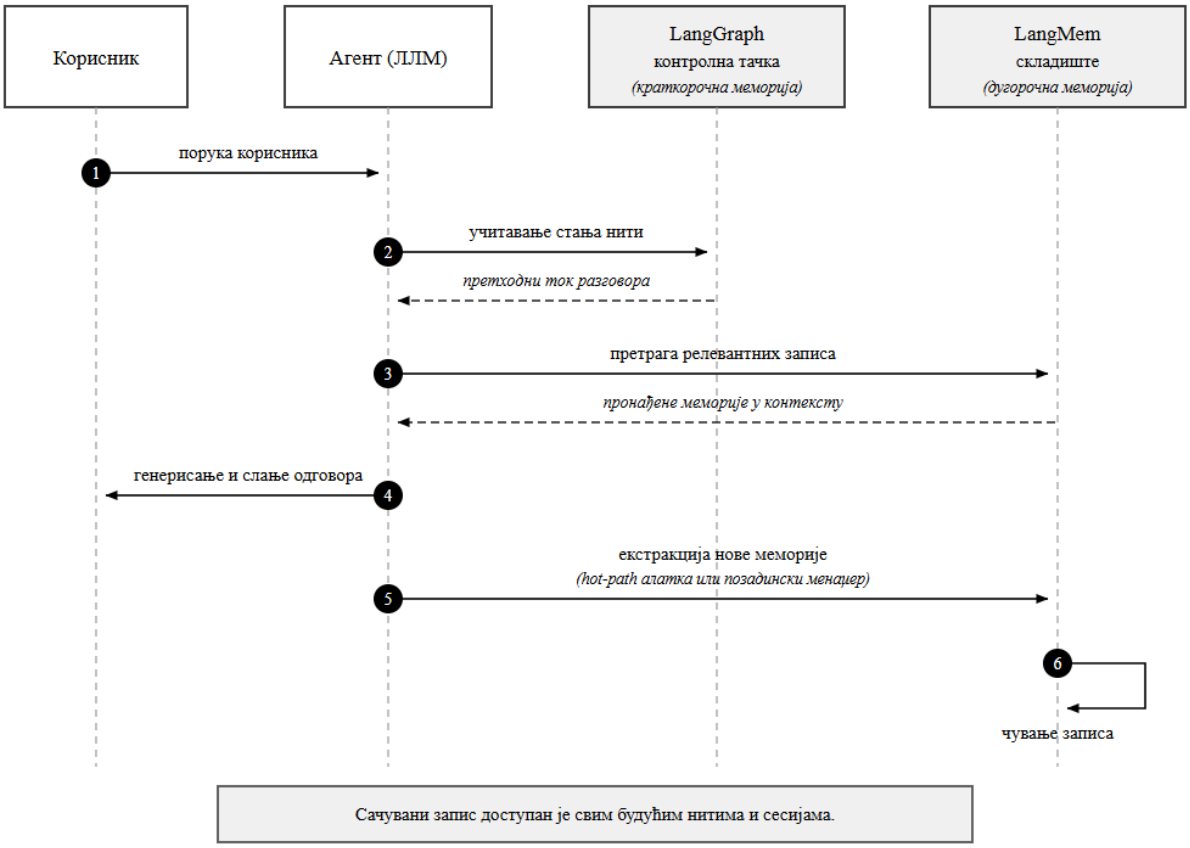
Поред експлицитне претраге коју агент сам покреће, меморију је могуће укључити у контекст и имплицитно, унутар припреме *prompt*-а, где систем може директно претражити складиште на основу последње поруке корисника и уградити пронађене записе у системску поруку пре него што се она уопште пошаље моделу [24]. Овакав приступ обезбеђује да агент увек располаже релевантним контекстом из меморије, без обзира да ли модел сам „одлучи” да га затражи.

Одржавање меморијских записа у *LangMem*-у ослања се на исте механизме

коришћене за њихово креирање – алатка за управљање меморијом и менаџер меморије подржавају не само додавање нових записа, него и ажурирање, консолидацију и брисање постојећих, у зависности од тога да ли нова информација допуњује, потврђује или противречи ономе што је већ сачувано [13]. **Именовани простори** при том омогућавају хијерархијску организацију меморије, на пример, одвајање по кориснику или апликацији, што олакшава циљано ажурирање само унутар релевантног дела складишта [23].

7.6. Интеграција LangMem SDK-а са LangGraph-ом

Иако *LangMem* функционише и потпуно независно од *LangGraph*-а, пуна функционалност долази до изражаја управо кроз нативну интеграцију са *LangGraph*-овим механизмом перзистенције – алатке за управљање и претрагу меморије граде се непосредно над *LangGraph*-овим *BaseStore* интерфејсом [23][24].



Слика 7.2. Ток извршавања у систему интегрисаном помоћу *LangGraph*-а и *LangMem*-а

Типичан ток извршавања у интегрисаном систему, приказан на слици 7.2., изгледа овако: агент прима поруку корисника → *LangGraph* учитава тренутно стање нити преко контролне тачке (краткорочна меморија) → *LangMem* претражује складиште (дугорочна меморија) и убацује релевантне записе у контекст → агент користи пронађене информације приликом генерисања одговора → *LangMem*, кроз „*hot-path*" алатку или позадински менаџер, издваја нову меморију из завршене интеракције → нова меморија се чува у складишту, доступна свим будућим нитима и сесијама [22][23][24].

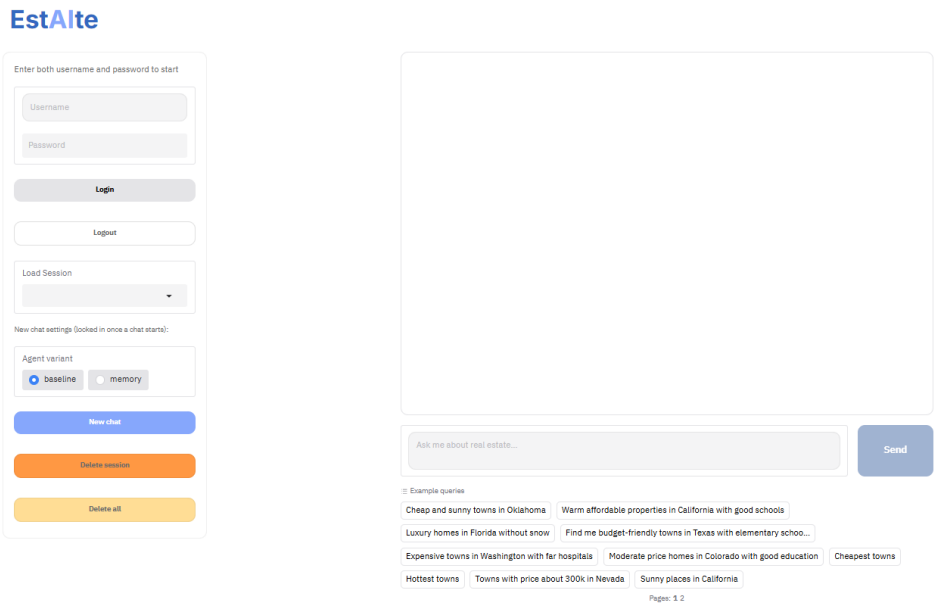
8. ПРОЈЕКТОВАЊЕ И ИМПЛЕМЕНТАЦИЈА СИСТЕМА СА ДУГОРОЧНОМ МЕМОРИЈОМ

Претходна поглавља описала су теоријске основе агентских система заснованих на великим језичким моделима, типове меморије и технологије LangGraph и LangMem SDK на општем, применљивом нивоу. Ово поглавље приказује конкретан систем изграђен у оквиру практичног дела овог мастер рада: апликацију за претрагу и препоруку градова погодних за пресељење, у којој је дугорочна меморија имплементирана управо помоћу *LangGraph*-а и *LangMem SDK*-а описаних у претходном поглављу.

Циљ поглавља је приказ пројектних одлука, архитектуре и кључних механизама који повезују теоријски оквир изложен у претходним поглављима са конкретним, функционалним системом. Приказано је коначно стање система, заједно са унапређењима уведеним у овом раду у односу на прототип из студијско-истраживачког рада; где је то битно, посебно је наведено шта је ново.

8.1. Дефинисање сценарија примене и захтева система

Као домен примене изабрана је претрага и препорука градова погодних за пресељење – задатак који природно захтева памћење информација о кориснику кроз више узастопних упита и, што је важније за тему овог рада, кроз више одвојених сесија разговора. Корисник у току конверзације износи преференце попут жељеног ценовног опсега некретнине, климатских услова, близине школа или болница, или изричито наводи шта не жели (нпр. одређену државу), а од агента се очекује да те преференце узме у обзир не само у оквиру исте сесије, него и приликом сваке наредне посете апликацији.

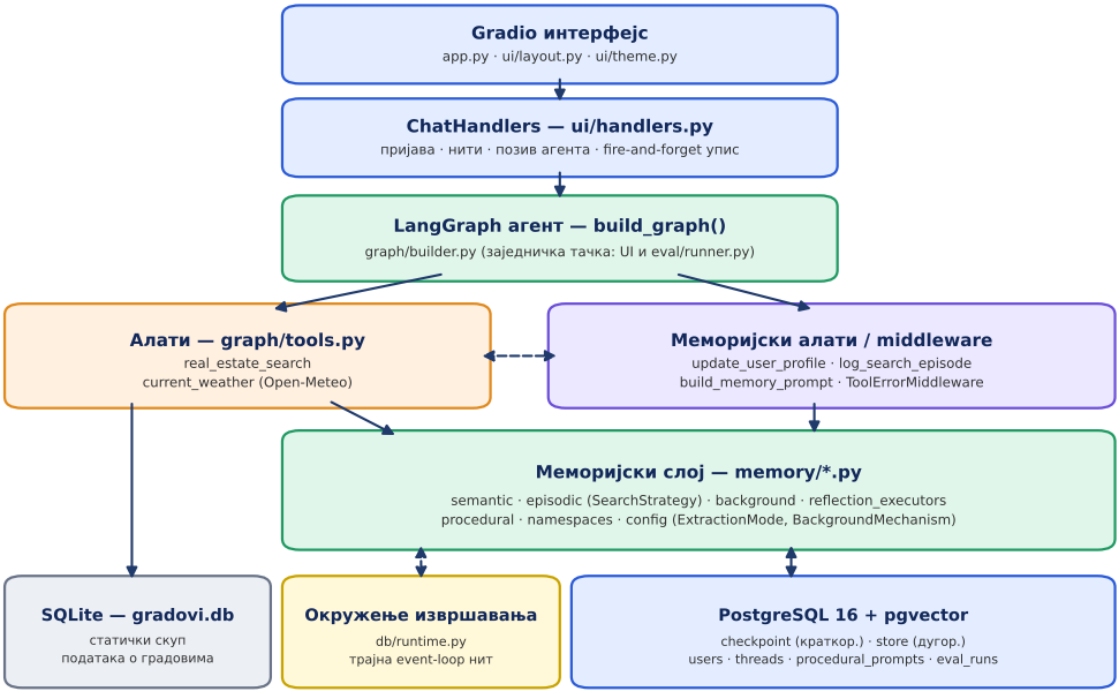


Слика 8.1. Кориснички интерфејс апликације у режиму без меморије (*baseline*)

Овакав задатак директно пресликава три раније описана типа меморије на конкретне податке: семантичка меморија чува трајне чињенице и преференце корисника (буџет, жељена клима, одбојност), епизодичка меморија бележи појединачне претраге као догађаје (који је упит постављен, који су резултати понуђени), док процедурална меморија, кроз механизам оптимизације системског *prompt*-а, треба да обезбеди да агент доследно поштује задата правила понашања (нпр. да увек наведе близину болница уз сваку препоруку), без обзира на то колико се системски *prompt* временом мења. На слици 8.1. приказан је интерфејс апликације.

8.2. Архитектура и компоненте система

Систем је организован слојевито, при чему сваки слој одговара једној од компоненти описаних на општем нивоу у поглављу 6. Улазна тачка је кориснички интерфејс изграђен помоћу библиотеке *Gradio*, преко кога корисник шаље поруке и бира параметре сесије. Изнад интерфејса налази се слој *ChatHandlers* (*ui/handlers.py*) који управља животним циклусом разговора – проналажењем или креирањем нити, позивањем агента и приказом одговора.



Слика 8.2. Архитектура имплементираног система

Средишњи део система чини *LangGraph* агент, изграђен функцијом *build_graph()* (*graph/builder.py*), која представља заједничку тачку улаза и за кориснички интерфејс и за скрипту за евалуацију, чиме је обезбеђено да се идентичан агент тестира и користи у пракси. Агент располаже алаткама за претрагу градова и прибављање тренутне временске прогнозе (*graph/tools.py*) и, у режимима са укљученом меморијом, додатним алаткама и *middleware* компонентама за рад са меморијом.

Агент и интерфејс деле трајно окружење извршавања (*db/runtime.py*). Приликом

покретања, *app.py* покреће једну трајну петљу догађаја у засебној нити и на њој отвара једну контролну тачку и једно складиште, који живе колико и сам процес. *Gradio* позива методе слоја *ChatHandlers* синхроно из сопствених радних нити, па функција *run_coroutine()* служи као мост ка тој заједничкој петљи. Ово решење је потребно због позадинског уписа. У прототипу се веза са базом отварала за сваку поруку и затварала чим се одговор врати. Пошто сада одговор више не чека да се упис заврши, веза би се затворила док позадински упис још траје и упис би пропао. Зато се веза отвара једном, при покретању апликације, и остаје отворена све време рада. Везе се отварају директно (*AsyncConnection*), а не преко менаџера контекста *from_conn_string()*, јер се код њих веза затвара чим *garbage collector* уништи генератор, што се у тестирању догађало у року од неколико секунди.

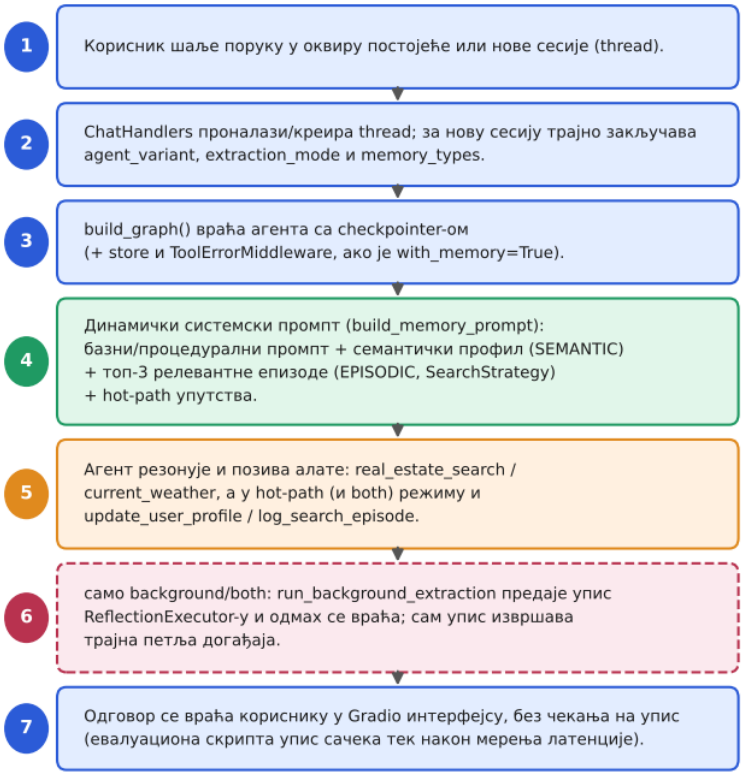
Прва алатка, *real_estate_search*, претражује локалну базу америчких градова на основу критеријума корисника (цена, клима, близина школа или болница) и враћа до пет најрелевантнијих резултата. Друга, *current_weather*, спољна је алатка која уживо дохвата тренутну временску прогнозу за конкретан град преко *Open-Meteo* сервиса (геокодирање града, па потом тренутна температура, стање неба и брзина ветра), и позива се искључиво када корисник експлицитно пита за тренутно/данашње временско стање, за разлику од историјских климатских података које већ обезбеђује *real_estate_search*. Алатке за рад са меморијом описане су у поглављу 8.6.

Меморијски слој (*memory/*.py*) имплементира семантичку, епизодичку и процедуралну меморију, ослањајући се на *LangMem SDK*. Модул *config.py* дефинише осе варијације система: *ExtractionMode* (*hot_path*, *background*, *both*), *MemoryType* (семантичка, епизодичка, процедурална), *SemanticBackgroundStrategy* (*fetch_merge*, *store_manager*) и *SearchStrategy* (*similarity*, *keyword*, *hybrid*). Подаци се трајно чувају у *PostgreSQL* бази са *pgvector* екстензијом за векторско индексирање, док статички скуп података о градовима остаје у одвојеној *SQLite* бази (*gradovi.db*), с обзиром на то да се та база не мења током рада система и не захтева семантичку претрагу.

Слика 8.2 приказује ову архитектуру у целини и та архитектура укључује и разлику, наслеђену директно из *LangGraph*-а, између контролне тачке, која чува краткорочно стање сваке нити, и складишта, које чува дугорочну меморију дељену између свих нити истог корисника [22].

8.3. Ток информација и механизам управљања меморијом

Слика 8.3 приказује типичан ток обраде једне корисничке поруке, конкретизујући раније описани општи ток (слика 7.2.) на ниво имплементираног система. По пријему поруке, слој *ChatHandlers* прво проналази или креира нит разговора; уколико је реч о потпуно новој сесији, у том тренутку се трајно фиксирају варијанта агента, режим екстракције меморије („*hot-path*”, „*background*” или „*both*”) и типови меморије који ће бити активни за целу ту сесију и ова одлука се не мења током трајања сесије.



Слика 8.3. Ток обраде корисничке поруке кроз слојеве система

Функција *build_graph()* затим враћа *LangGraph* агента опремљеног контролном тачком (увек) и складиштем (само ако је меморија укључена за ту сесију), као и компонентом *ToolErrorMiddleware* која збрињава грешке алатке за ажурирање профила (поглавље 8.10). Динамичка функција *build_memory_prompt()* конструише системски *prompt* „у лету”, спајајући или базни или процедурално оптимизовани *prompt* са прибављеном меморијом: тренутним семантичким профилом корисника (ако је тип *SEMANTIC* активан), блоком „*RELEVANT PAST SEARCHES*” са највише три епизоде релевантне за последњу поруку корисника (ако је активан тип *EPISODIC*) и, по потреби, упутствима за коришћење „*hot-path*” алатки. Прибављање је независно од режима екстракције, а алатке се у *prompt*-у помињу само ако су заиста везане. Свим режимима са семантичком меморијом заједничко је правило о коришћењу профила (*SEMANTIC_READ_RULE*): профил се користи при претрази, од корисника се не тражи да понавља оно што профил већ садржи, а ако неке преференце недостају, претражује се са оним што је познато. Упутство за упис профила (*SEMANTIC_TOOL_USAGE_HINT*) додаје се само у режимима „*hot-path*” и „*both*”; његов утицај на понашање агента испитан је у поглављу 9.5.

Након што агент резонује и, по потреби, позове алатке за претрагу градова или временску прогнозу, следи кључна разлика између механизама екстракције. У „*hot-path*” режиму меморија се уписује током самог резоновања, кроз алатке које агент сам позива. У „*background*” режиму упис се покреће након што је одговор генерисан: *ChatHandlers* позива *run_background_extraction()*, која само планира задатке и враћа списак покренутих асинхроних задатака, док интерфејс одмах приказује одговор, а задатке препушта трајној петљи. У прототипу се на овај позив чекало пре враћања одговора, па је кашњење у потпуности падало на корисника. У режиму „*both*” оба механизма делују над истом поруком.

8.4. Избор технологија и окружења

Апликација је реализована у програмском језику *Python*, уз *LangGraph* и *LangMem SDK* као основу за агента и меморију, и *Gradio* за кориснички интерфејс, изабран због брзине израде интерактивних прототипова уз задржавање пуне контроле над изгледом и понашањем компоненти. Верзије језика и кључних библиотека, са којима су спроведене и евалуације описане у поглављу 9, наведене су у табели 8.1. *LangMem SDK* је у време израде рада у верзији 0.0.x, па се његов интерфејс може мењати између издања, због чега је навођење тачних верзија важно за поновљивост резултата.

За перзистенцију је изабран *PostgreSQL 16* са *pgvector* екстензијом, покренут као засебан контејнер помоћу *Docker Compose*-а. Ова база истовремено служи и као *LangGraph*-ова контролна тачка (кроз *AsyncPostgresSaver*) и као складиште за дугорочну меморију (кроз *AsyncPostgresStore*) [22], поред чега чува и помоћне табеле апликације – кориснике, нити разговора, процедурално оптимизоване *prompt*-ове и резултате евалуационих сесија. За статичан скуп података о градовима (цене некретнина, клима, близина школа и болница) коришћена је, из практичних разлога, посебна *SQLite* база (*gradovi.db*), с обзиром на то да ти подаци не захтевају ни векторску претрагу ни честа ажурирања.

Сви позиви језичког модела у систему користе *OpenAI*-јев модел *gpt-4o-mini*, који има контекстни прозор од 128.000 токена, највише 16.384 излазна токена по одговору и границу знања из октобра 2023. године [30]. Модел се позива по имену *gpt-4o-mini*, без навођења датираног снимка; једини објављени датирани снимак овог модела је *gpt-4o-mini-2024-07-18* [30]. Исти модел користи се у свим улогама, уз различите температуре: за резоновање агента 0,2, за анализу упита у алатки за претрагу градова и за позадинску екстракцију семантичке и епизодичке меморије 0,1, а за оптимизацију процедуралног *prompt*-а 0,0. Векторске репрезентације меморијских записа добијају се моделом *text-embedding-3-small* (1536 димензија).

Компонента	Верзија	Улога у систему
Python	3.14	језик имплементације
LangGraph (langgraph)	1.2.11	граф агента, контролна тачка и складиште
langgraph-checkpoint-postgres	3.1.2	*AsyncPostgresSaver* и *AsyncPostgresStore*
LangChain (langchain)	1.4.0	*create_agent* и *middleware* компоненте
langchain-openai	1.1.8	веза са *OpenAI* моделима
LangMem SDK (langmem)	0.0.30	алатке и менаџер меморије, *ReflectionExecutor*, оптимизатор *prompt*-а
Gradio	6.5.1	кориснички интерфејс
psycopg	3.3.5	веза са *PostgreSQL* базом
PostgreSQL са *pgvector* екстензијом	16	трајно складиште и векторски индекс (*Docker* слика pgvector/pgvector:pg16)

Табела 8.1. Верзије језика, библиотека и окружења коришћених у раду

8.5. Имплементација агента и интеграција LangGraph-а и LangMem SDK-а

Слика 8.4. приказује поједностављен одломак функције *build_graph()*, која представља тачку у којој се теоријски концепти претварају у конкретан, извршив агент. Функција прима параметре сесије (да ли је меморија укључена, режим екстракције, листу активних типова меморије) и, у зависности од њих, саставља одговарајући скуп алатки и *middleware* компоненти пре него што позове *create_agent()* и врати компајлирани граф [21], повезан са *AsyncPostgresSaver* контролном тачком и, када је меморија укључена, *AsyncPostgresStore* складиштем.

```python
def build_graph(
    *,
    with_memory: bool,
    extraction_mode: ExtractionMode | None = None,
    memory_types: frozenset[MemoryType] | None = None,
    search_strategy: SearchStrategy = SearchStrategy.SIMILARITY,
    checkpointer: BaseCheckpointSaver,
    store: BaseStore | None = None,
    user_id: int,
    search_service: RealEstateSearchService,
    llm_model: str = "gpt-4o-mini",
):
    tools = [make_real_estate_tool(search_service), make_current_weather_tool()]

    if not with_memory:
        graph = create_agent(
            model=ChatOpenAI(model=llm_model, temperature=0.2),
            tools=tools,
            system_prompt=BASE_SYSTEM_PROMPT,
            checkpointer=checkpointer,
            store=store,
        )
        return graph

    if store is None:
        raise ValueError("build_graph(with_memory=True, ...) requires a store")

    active_types = memory_types if memory_types is not None else ALL_MEMORY_TYPES

    if extraction_mode in (ExtractionMode.HOT_PATH, ExtractionMode.BOTH):
        if MemoryType.SEMANTIC in active_types:
            tools.append(make_semantic_memory_tool(user_id, store))
        if MemoryType.EPISODIC in active_types:
            tools.append(make_episodic_memory_tool(user_id, store))

    graph = create_agent(
        model=ChatOpenAI(model=llm_model, temperature=0.2),
        tools=tools,
        middleware=[
            build_memory_prompt(user_id, extraction_mode, active_types, search_strategy),
            ToolErrorMiddleware(_on_update_user_profile_error, tools=["update_user_profile"]),
        ],
        checkpointer=checkpointer,
        store=store,
    )
    return graph
```

Слика 8.4. Одломак кода функције *build_graph()*

Ова заједничка функција користи се и у *Gradio* интерфејсу и у *eval/runner.py* скрипти за евалуацију, чиме је обезбеђено да агент који се оцењује буде потпуно идентичан ономе који користи крајњи корисник, без посебне, „тест" верзије агента.

У односу на прототип, функција *build_graph()* прима и параметар *search_strategy*, везује алатке за упис и у режиму „*both*" и уводи компоненту *ToolErrorMiddleware*; слика 8.4. приказује ажурирану верзију функције.

8.6. Аутоматска екстракција и перзистентно складиштење меморије

Семантичка меморија имплементирана је помоћу *create_manage_memory_tool* алатке у „*hot-path*" режиму [24], приказано на слици 8.5., односно помоћу *create_memory_store_manager* менаџера у „*background*" режиму [25]. Профил корисника структуриран је као именовани запис у складишту и обухвата поља попут жељеног ценовног опсега некретнине, преферираних климатских услова, близине школа или болница, и изричито наведених нетрпељивости (нпр. државе или региона које корисник

жели да избегне) – управо оне податке који се проверавају у тест сценаријима.

Позадинско ажурирање семантичког профила у унапређеном систему подразумевано користи детерминистичку стратегију *fetch_merge* (*aupdate_profile_from_messages()*): постојећи профил учита се по познатом идентификатору, ЛЛМ са структурираним излазом (*UserProfile*) враћа потпуно ажурирани профил, који се уписује назад под истим кључем. Разлог је понашање менаџера *create_memory_store_manager* из прототипа: његова претрага по сличности понекад не пронађе постојећи профил и упише други документ уместо спајања. Тај пут је задржан као алтернатива (*store_manager*) да би могао да се евалуира. Планирање уписа обавља се независно од тога шта се уписује, помоћу *ReflectionExecutor*-а, који уклања застареле задатке за исту нит (*debounce*) и подржава одложено извршавање [27]. Како *fetch_merge* није *Runnable*, за њега је написан адаптер (*_FetchMergeReflector*), а извршиоци се кеширају по врсти, кориснику и стратегији, јер сваки при конструкцији покреће радну нит.

```python
import uuid
from typing import Optional, Tuple

from langchain_openai import ChatOpenAI
from langgraph.store.base import BaseStore
from langmem import create_manage_memory_tool, create_memory_store_manager

from memory.namespaces import semantic_ns
from memory.schemas import UserProfile
from memory.stamped_store import KindStampingStore

SEMANTIC_TOOL_INSTRUCTIONS = (
    "This namespace holds exactly ONE user profile document, not a list of separate "
    "memories. The system prompt shows you the CURRENT PROFILE and its id (if one "
    "exists). If an id is shown, ALWAYS call this tool with action='update' and that "
    "same id — never create a second profile. Only use action='create' if the system "
    "prompt says no profile exists yet for this user. You MUST pass a `content` object "
    "with the relevant fields filled in (merging with what's already known) — never "
    "omit content or leave it null."
)


def make_semantic_memory_tool(user_id: int, store: BaseStore):
    return create_manage_memory_tool(
        namespace=semantic_ns(user_id),
        schema=UserProfile,
        actions_permitted=("create", "update"),
        store=KindStampingStore(store, UserProfile.__name__),
        instructions=SEMANTIC_TOOL_INSTRUCTIONS,
        name="update_user_profile",
    )
```

```python
from langgraph.store.base import BaseStore, SearchItem
from langmem import create_manage_memory_tool

from memory.config import SearchStrategy
from memory.namespaces import episodic_ns
from memory.schemas import SearchEpisode
from memory.stamped_store import KindStampingStore

_CANDIDATE_POOL_SIZE = 50

EPISODIC_TOOL_INSTRUCTIONS = (
    "Call this after you have shown the user real_estate_search results, to log what "
    "was searched for and found. Each call creates a NEW, independent record — unlike "
    "the user profile, this is a growing history, not a single document to update. "
    "Do not call action='update' or action='delete'. You MUST pass a `content` object "
    "with `query`, `summary`, and `top_result_towns` filled in — never omit content "
    "or leave it null."
)


def make_episodic_memory_tool(user_id: int, store: BaseStore):
    return create_manage_memory_tool(
        namespace=episodic_ns(user_id),
        schema=SearchEpisode,
        actions_permitted=("create",),
        store=KindStampingStore(store, SearchEpisode.__name__),
        instructions=EPISODIC_TOOL_INSTRUCTIONS,
        name="log_search_episode",
    )
```

```python
from langchain_openai import ChatOpenAI
from langmem import Prompt, create_prompt_optimizer
from langmem.prompts.types import AnnotatedTrajectory

from db.procedural_repo import get_latest_procedural_prompt, insert_procedural_prompt
from graph.prompts import BASE_SYSTEM_PROMPT

UPDATE_INSTRUCTIONS = (
    "Make minimal, targeted changes to how the assistant should search, log "
    "memories, and respond, based only on the observed conversations. It must "
    "remain a system prompt for a real estate search assistant that calls the "
    "real_estate_search tool — do not remove that instruction."
)
WHEN_TO_UPDATE = (
    "Whenever the trajectories reveal a recurring mistake (skipped tool calls, "
    "ignored user preferences, unhelpful answers) or a missed opportunity to "
    "serve the user better."
)


def make_prompt_optimizer(llm_model: str = "gpt-4o-mini"):
    return create_prompt_optimizer(
        ChatOpenAI(model=llm_model, temperature=0.0),
        kind="gradient",
    )
```

Слика 8.5. Одломци кода алатака за управљање типовима меморија

Режим „*both*” комбинује оба пута. Семантички упис безбедно се комбинује, јер позадински пролаз спаја профил преко онога што је „*hot-path*” алатка већ уписала, док се епизодички позадински пролаз у овом режиму прескаче: менаџер записује и чита записе са пољем „*kind*”, а „*hot-path*” алатка је обичне записе уписивала без њега, па је менаџер при читању падао са грешком *KeyError('kind')* – понашање које се поновило у 4 од 4 покушаја. Тај узрок је отклоњен омотачем *KindStampingStore*

(*memory/stamped_store.py*), који свакој вредности коју алатке за меморију уписују додаје поље „*kind*”, па су записи оба пута међусобно компатибилни. Епизодички позадински пролаз у режиму „*both*” ипак је остао прескочен, уз претпоставку да епизоду бележи „*hot-path*” механизам. Евалуација је, међутим, показала да агент са коначним *prompt*-ом у порукама у којима уписује профил епизоду по правилу не бележи (поглавље 9.5), па се у овом режиму губе управо епизоде из порука које доносе нове податке о кориснику; поновно укључивање овог пролаза остаје правац даљег рада.

Епизодичка меморија бележи сваку претрагу као засебан запис (упит корисника, кључни резултати, временска ознака), док процедурална меморија, преко оптимизатора *prompt*-а из *LangMem SDK*-а, повремено преиспитује историју разговора и предлаже измене базног системског *prompt*-а, у циљу доследнијег понашања агента.

Именовани простори (*namespaces*) организовани су по кориснику, чиме је обезбеђена изолација меморије – меморија једног корисника никада није видљива другом, нити се меша са статичким скупом података о градовима.

8.7. Структура базе података и физички облик меморијских записа

Иако је претходно поменуто да се сви подаци трајно чувају у *PostgreSQL* бази са *pgvector* екстензијом, вредно је детаљније објаснити како та база заправо изгледа изнутра и у ком тачно облику се меморијски записи у њој налазе. Реч је о једној, физички истој *Postgres* бази која истовремено служи трима различитим сврхама: *LangGraph* самостално креира и одржава сопствене табеле за контролну тачку (кроз *AsyncPostgresSaver.setup()*) и за складиште дугорочне меморије (кроз *AsyncPostgresStore.setup()*), док апликација засебно дефинише четири сопствене табеле, наведене у *db/schema.sql*, приказане на слици 8.6. и креиране идемпотентно приликом покретања *db/migrate.py*: *users* (кориснички налози), *threads* (регистар нити разговора са варијантом агента, режимом екстракције и активним типовима меморије закљученим за целу сесију), *procedural_prompts* (верзионисана историја оптимизованих системских *prompt*-ова по кориснику) и *eval_runs* (лог резултата сваке евалуационе сесије, укључујући измерене метрике у *JSONB* колони).

Складиште које одржава *LangGraph* (*AsyncPostgresStore()*) функционише као генерички документни магацин, а не као традиционална релациона табела с унапред дефинисаним колонама: сваки запис идентификован је именованим простором (*namespace*) – низом текстуалних сегмената попут („*memories*”, <*user_id*>, „*semantic_profile*”) – и кључем, док је садржај записа произвољан *JSON* документ. Приликом уписа сваком запису се, на основу конфигурације *STORE_INDEX_CONFIG* (*db/connection.py*), додаје и векторска репрезентација добијена преко *OpenAI*-јевог модела *text-embedding-3-small* (1536 димензија), коју *pgvector* екстензија индексира ради семантичке претраге по сличности. Ова инфраструктура за векторску претрагу активно се користи: поред уписа, индекс служи за претрагу епизодичке меморије по сличности (поглавље 8.8).

Именовани простори у овом систему организовани су по кориснику и по типу меморије (*memory/namespaces.py*), при чему свака од три врсте меморије физички изгледа другачије. Семантичка меморија чува се као тачно један, стално ажуриран *JSON* документ по кориснику, у простору („*memories*”, <*user_id*>, „*semantic_profile*”), чија поља одговарају *Pydantic* шеми *UserProfile* – жељени ценовни опсег, преферирана клима, близина болница и школа, изричите нетрпељивости и слободан текст напомена; сваки нови упис замењује претходну вредност истог документа (*fetch-merge-write*),

никада не додаје нови. Епизодичка меморија, насупрот томе, чува се као растућа колекција независних *JSON* докумената у простору („*memories*”, <*user_id*>, „*episodic_search*”), при чему сваки документ (шема *SearchEpisode*: упит, кратак резиме и списак понуђених градова) добија сопствени, аутоматски генерисани идентификатор – ништа се не ажурира нити брише, само додаје. Процедурална меморија, коначно, физички није ни документ у овом складишту: њено меродавно чување остварено је кроз обичну релациону табелу *procedural_prompts*, где свака нова, оптимизована верзија системског *prompt*-а постаје нови ред (са редним бројем верзије и списком нити разговора из којих је изведена); истоимени простор („*memories*”, <*user_id*>, „*procedural_prompt*”) у *LangGraph* складишту постоји само као опциона копија, не и као примарни извор истине.

db > schema.sql > ...

```sql
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    username TEXT UNIQUE NOT NULL,
    display_name TEXT,
    password_hash TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

ALTER TABLE users ADD COLUMN IF NOT EXISTS password_hash TEXT;

CREATE TABLE IF NOT EXISTS threads (
    thread_id UUID PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id),
    title TEXT,
    agent_variant TEXT NOT NULL,
    extraction_mode TEXT,
    memory_types TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

ALTER TABLE threads ADD COLUMN IF NOT EXISTS memory_types TEXT;

CREATE TABLE IF NOT EXISTS procedural_prompts (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id),
    version INTEGER NOT NULL,
    prompt_text TEXT NOT NULL,
    source_thread_ids UUID[],
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (user_id, version)
);

CREATE TABLE IF NOT EXISTS eval_runs (
    id SERIAL PRIMARY KEY,
    scenario_name TEXT NOT NULL,
    agent_variant TEXT NOT NULL,
    extraction_mode TEXT,
    user_id INTEGER NOT NULL REFERENCES users(id),
    thread_ids UUID[] NOT NULL,
    metrics_json JSONB NOT NULL,
    run_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Слика 8.6. Одломак *SQL* шеме перзистентног складишта

Заједничко свим трима типовима меморије јесте да је идентификатор корисника уграђен непосредно у сам именовани простор, чиме је изолација података остварена на нивоу самог кључа записа – упит над складиштем ограничен на простор једног корисника структурно не може вратити ништа што припада другом кориснику. Овакво груписање по кориснику потпуно је одвојено од статичког скупа података о градовима (*gradovi.db*), који се налази у засебној *SQLite* бази и уопште не пролази кроз *LangGraph* складиште, с обзиром на то да се не мења током рада система нити захтева персонализацију.

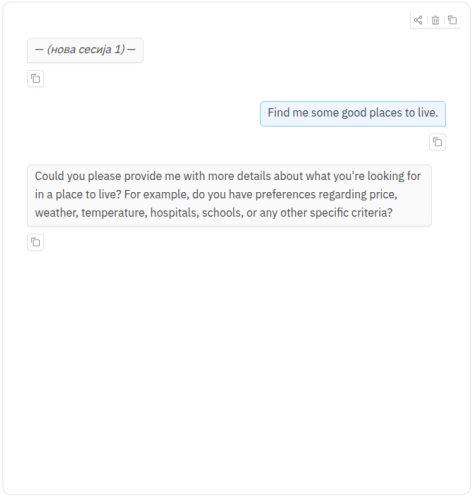
8.8. Претрага, прибављање и коришћење меморије у генерисању одговора

Прибављање меморије у имплементираном систему остварено је имплицитно, кроз динамичку функцију *build_memory_prompt()*: пре сваког позива агента, у системску поруку се уграђују (1) тренутни семантички профил корисника, ако је активан тип *SEMANTIC*, и (2) блок „*RELEVANT PAST SEARCHES*” са највише три епизоде, ако је активан тип *EPISODIC*. Оба прибављања независна су од режима екстракције: и „*hot-path*” и „*background*” агент читају исту меморију. Профил у *prompt*-у сам по себи не говори агенту да га треба применити, па је у системски *prompt* свих режима са семантичком меморијом додато правило о његовом коришћењу (*SEMANTIC_READ_RULE*), чија је потреба показана у поглављу 9.5.

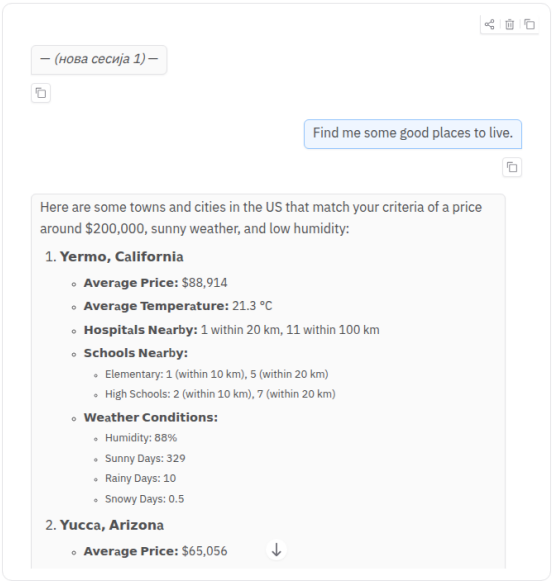
У прототипу из студијско-истраживачког рада епизодичка меморија била је само уписна: постојала је алатка за уписивање епизода, али ниједна компонента није учитавала раније уписане епизоде назад у контекст агента. Зато варијанта само са епизодичком меморијом у почетној евалуацији није показала никакво побољшање у односу на „*baseline*” (поглавље 9.3). У овом раду то ограничење је отклоњено.

Функција *asearch_episodes()* (*memory/episodic.py*) прибавља епизоде корисника релевантне за последњу поруку, при чему се стратегија рангирања бира параметром *SearchStrategy*. Стратегија *SIMILARITY* користи векторску претрагу самог складишта (*store.asearch(query=...)*) над *pgvector* индексом. Стратегија *KEYWORD* рангира епизоде према уделу речи упита које се јављају у тексту епизоде (поља *query* и *summary*), без уграђивања, па хвата тачна подударања која претрага по сличности може да промаши. Стратегија *HYBRID* рангира по збиру 0,5 · сличност + 0,5 · преклапање кључних речи. Стратегије *KEYWORD* и *HYBRID* прибављају кандидатски скуп од највише 50 епизода и саме га рангирају, јер *BaseStore* нема изворну лексичку претрагу. У *prompt* се убацују највише три епизоде, а сваки сажетак се скраћује на 220 карактера, да би додатни контекст остао у границама неколико стотина токена – у складу са налазима из поглавља 3 о цени предугог контекста.

Слике 8.7. и 8.8. илуструју разлику коју прибављена (семантичка) меморија прави у пракси, на примеру исте корисничке преференце изнете у једној сесији, а поново упитане у наредној: без меморије агент тражи да корисник понови детаље, док уз укључену меморију агент самостално користи раније сачуван профил.



Слика 8.7. Агент без меморије (*baseline*) не памти преференцу изнету у претходној сесији



Слика 8.8. Агент са укљученом меморијом самостално користи преференцу из претходне сесије

8.9. Ажурирање меморије током више сесија

Ажурирање семантичке меморије реализовано је по принципу „учитај – спој – сачувај" (*fetch-merge-write*): приликом сваког уписа, систем прво учитава постојећи профил корисника из складишта, спаја га с новоизвученом информацијом (нова вредност замењује стару уколико долази до директне противречности, нпр. промене буџета), а затим чува ажурирани запис назад, уместо да једноставно дода нови, потенцијално противречан запис поред старог.

На пример, ако корисник у једном тренутку наведе да му је буџет око 200.000 долара, а нешто касније, у истој или новој сесији, то измени на 300.000 долара, накнадна препорука градова заснива се искључиво на ажурираној вредности – стара вредност се не помиње нити утиче на препоруку.

Ажурирање се односи искључиво на семантичку меморију – с обзиром на то да епизодичка меморија бележи појединачне, временски означене догађаје (а не тренутно стање), нови упис не замењује претходне епизоде, већ им се придружује, што одговара њеној намени.

8.10. Отпорност на грешке и аутоматско тестирање

Да би ажурирао постојећи профил, агент алатку *update_user_profile* позива са *action='update'* и идентификатором профила, који види у *prompt*-у. Током евалуације прототипа уочено је да модел понекад изостави тај идентификатор, што изазива грешку која је прекидала читав ток. У прототипу је заштита постојала само у скрипти за евалуацију, док је апликација остајала рањива. У овом раду заштита је уграђена у самог агента: компонента *ToolErrorMiddleware*, везана искључиво за алатку

update_user_profile, хвата ову грешку и уместо прекида враћа агенту поруку алатке. Прва верзија те поруке говорила је агенту да настави без чувања, па он није понављао позив и упис се губио (поглавље 9.5). Порука зато сада налаже поновни позив са исправним аргументима: са *action='create'* ако профил још не постоји, односно са идентификатором из *prompt*-а ако постоји. Аутоматски поновни позив са истим аргументима не би помогао, јер је грешка детерминистичка, а не пролазна. Пошто и интерфејс и скрипта за евалуацију граде агента истом функцијом *build_graph()*, заштита важи на оба места.

Поред евалуације, пројекат садржи шеснаест аутоматских јединичних тестова (*tests/*, *pytest* са *asyncio_mode = auto*) који не захтевају ни мрежу, ни базу, ни ЛЛМ. Три теста проверавају алатку *current_weather* над *HTTP* одговорима (избор града по савезној држави, извештај о прогнози и непознат град). Четири теста проверавају позадинску екстракцију: да поновљено *fetch-merge-write* ажурирање увек даје тачно један профил по кориснику (над *InMemoryStore* и лажним ЛЛМ-ом), да подразумевана стратегија позадински задатак предаје као *fetch_merge*, да се може изабрати стратегија *store_manager* и да режим „*both*" прескаче епизодички пролаз (над лажним извршиоцима). Пет тестова проверава *ToolErrorMiddleware* над лажним моделом који изводи позив алатке (случајеви изостављеног и празног идентификатора и обрада изузетака). Четири теста проверавају строгу метрику *memory_accuracy_strict*: одговор који само понавља излаз алатке („*Schools nearby*", „*Humidity*") добија 0, прави опозив добија 1, и сваки сценарио декларише дискриминативне групе. Свих шеснаест тестова пролази.

Евалуациони логови су проширени ради анализе узрока: за сваки корак бележе се позиви алатки са аргументима, резултати алатки, епизоде које је прибављање вратило и садржај профила одмах након уписа (*turn_logs* у *JSON* транскриптима). Управо ови логови омогућили су налазе из поглавља 9.4 и 9.5.

9. ЕКСПЕРИМЕНТАЛНА ЕВАЛУАЦИЈА И АНАЛИЗА РЕЗУЛТАТА

Ово поглавље приказује резултате експерименталне провере система описаног у претходном поглављу. Циљ евалуације није поређење са другим, спољашњим системима, нити академско бенчмарковање у ужем смислу, већ провера да ли имплементирани механизми дугорочне меморије заиста остварују очекивано понашање – да ли агент памти оно што треба, да ли то ради доследно кроз више сесија, и по коју цену у погледу времена одговора.

У ту сврху израђен је посебан скуп скрипти за евалуацију (*eval/*), независан од корисничког интерфејса, али заснован на истом *build_graph()* агенту описаном раније, чиме је обезбеђено да се оцењује стварно понашање система, а не поједностављена симулација.

Евалуација је спроведена у две фазе. Прво је евалуиран прототип из студијско-истраживачког рада, а његови резултати задржани су као референтна тачка. Затим је, након унапређења описаних у поглављу 8, евалуиран унапређени систем: најпре циљаним проверама током развоја, а потом двема потпуним евалуацијама – са првобитном и са коначном верзијом *prompt*-а. Да би у сваком тренутку било јасно на који систем се неки резултат односи, у наставку се користе ознаке из табеле 9.1: „почетни резултати” за прототип и „завршни резултати” за унапређени систем са коначним *prompt*-ом.

Ознака у тексту	Систем	Сценарији	Понављања	Где су резултати
Почетни резултати	прототип из студијско-истраживачког рада	4	*n = 5*	поглавља 9.2 и 9.3
Циљане провере	унапређени систем током развоја	1–2	*n = 1* до 5	поглавља 9.4 и 9.5
Прва евалуација унапређеног система	унапређени систем, првобитни *prompt*	6	*n = 5*	поглавље 9.5 (табела 9.8)
Завршни резултати	унапређени систем, коначни *prompt*	6	*n = 5*	поглавље 9.4

Табела 9.1. Преглед евалуација и ознаке резултата коришћене у овом поглављу

9.1. Методологија евалуације и дефинисање тест сценарија

Дефинисано је шест тест сценарија (*eval/scenarios.py*), приказаних у табели 9.2. Сваки сценарио проверава другачији аспект памћења – од основног преношења преференце, преко истрајности кроз више одвојених сесија, до отпорности на ирелевантне информације и разликовања више претрага у епизодичкој меморији. Почетна евалуација обухватила је прва четири сценарија, а последња два, посвећена епизодичкој меморији, уведена су у унапређеном систему.

Сценарио	Опис	Шта се тестира
Буџет и клима	Корисник у првој сесији наведе жељени буџет и климатске услове, а затим у новој сесији, без	Преношење експлицитно наведене преференце

	понављања детаља, тражи препоруку градова.	
Држава и школа	Слично претходном (преференце у првој, упит у новој сесији), али са преференцама везаним за државу и близину школа.	Генерализација на други тип преференце
Нетрпељивост (3 сесије)	Корисник у првој сесији наведе шта не жели (снег и хладне зиме), а то се проверава у две потпуно нове, одвојене сесије.	Истрајност меморије кроз више сесија
Дистрактор	Корисник у првој сесији наведе буџет и жељену државу (Орегон), уз ирелевантну информацију (рођак се преселио у Вермонт), а у новој сесији тражи препоруку градова.	Отпорност на лажно памћење
Опозив епизоде	Корисник у првој сесији наведе разлог претраге (ћерка почиње студије код Финикса), а у новој сесији пита шта је навео као разлог претходне претраге.	Прибављање епизодичке меморије (нови сценарио)
Разликовање две епизоде	Корисник у првој сесији тражи јефтине градове у Аризони уз разлог (ћерка почиње студије код Финикса), у другој тражи градове у Орегону уз други разлог (отварање пекаре), а у трећој пита због чега је тражио у Орегону.	Прибављање праве од две епизоде, без мешања са другом (нови сценарио)

Табела 9.2. Преглед тест сценарија коришћених у евалуацији

Сваки сценарио извршен је у више варијанти агента, груписаних у две осе поређења. Прва је оса механизма екстракције: агент без меморије („*baseline*”) пореди се са агентима код којих су активне и семантичка и епизодичка меморија, а који меморију уписују у режиму „*hot-path*”, „*background*” или „*both*” (режим „*both*” уведен је тек у унапређеном систему). Друга је оса типа меморије: поред „*baseline*”-а, тестирају се варијанте само са семантичком („*semantic_only*”) и само са епизодичком меморијом („*episodic_only*”), свака у свим режимима екстракције. Тиме се одвојено мери утицај начина уписа и утицај појединачног типа меморије. Свака комбинација сценарија и варијанте понављана је пет пута ($n = 5$), са новим корисником у сваком понављању, а наводе се средња вредност и (у табелама 9.3 и 9.5) стандардна девијација, због природне варијабилности одговора великог језичког модела.

Скрипта поред метрика бележи и логове по кораку – позиве алатки, прибављене епизоде и садржај профила након уписа – на основу којих је могуће утврдити зашто је одређени резултат добијен.

Конкретно, евалуација треба да одговори на следећа питања:

- Да ли агент уопште упамти оно што треба да упамти, односно да ли исправно прибавља и користи сачувану информацију (тачност прибављене меморије)?
- Да ли то ради доследно кроз више одвојених сесија, а не само повремено, случајно (конзистентност понашања)?
- По коју цену у времену одговора долази то памћење, у односу на агента без меморије (латенција и додатно кашњење)?
- Да ли агент „измишља” преференце из ирелевантних информација, односно лажно памти нешто што корисник никада није рекао (стопа лажног памћења)?

За сваку комбинацију мере се следеће метрике:

- строга тачност прибављене меморије (***memory_accuracy_strict***);
- конзистентност понашања кроз сесије (***behavioral_consistency***) – удео дискриминативних група маркера које, када се једном појаве у одговору, остају присутне у одговору сваке наредне сесије, израчунат за свако понављање посебно и усреднјен по понављањима (поглавље 9.6);

- стопа лажног памћења (***false_memory_rate***);
- просечно време одговора (***avg_latency_seconds***);
- додатно кашњење у односу на „*baseline*” варијанту истог сценарија (***latency_overhead_vs_baseline_seconds***);
- тачност прибављања епизоде, односно да ли најбоља прибављена епизода садржи очекивани маркер у пољу *query* или *summary* (***episodic_retrieval_accuracy***).

Тачност се мери тако што се проверава да ли одговор агента садржи очекиване маркере, као што су износ буџета или назив државе. Првобитна метрика (*memory_accuracy*), коришћена у почетној евалуацији, узимала је у обзир све маркере. Накнадном анализом сачуваних транскрипата утврђено је да се поједини маркери налазе и у стандардном излазу алатке за претрагу градова – на пример „*school*” у пољу „*Schools nearby*” и „*humid*” у пољу „*Humidity*” – па их агент наводи и без меморије. Зато је уведена строга тачност (*memory_accuracy_strict*), која узима у обзир само маркере које излаз алатке не садржи, као што су буџет и називи држава. Обе метрике мере присуство очекиваних података у одговору, а не укупни квалитет препоруке. За почетне резултате строга тачност накнадно је израчуната из сачуваних транскрипата, без нових позива модела, а завршни резултати наводе се само по строгој метрици.

Задатак рада предвиђа мерење тачности прибављених меморија, што за два типа меморије има различит смисао. Прибављање семантичке меморије у имплементираном систему је детерминистичко: профил корисника је један документ са познатим кључем, који се, ако постоји, у целини уграђује у системски *prompt* (поглавље 8.8). Прибављање профила је зато тачно чим је профил уписан, па се провера семантичке меморије своди на две друге карике – да ли је профил исправно уписан и да ли га агент користи. Строга тачност обухвата обе карике заједно, јер се запамћени податак може појавити у одговору у новој сесији само ако је уписан и ако га је агент искористио. Да би се оне раздвојиле, из логова по кораку накнадно су, без нових позива модела (скрипта *eval/memory_usage.py*), израчуната још два показатеља: удео дискриминативних група маркера садржаних у профилу на крају прве сесије (упис) и удео група које се у новој сесији појављују у упиту алатке за претрагу градова (коришћење у претрази). Строга тачност бодује податак у одговору и када га агент није употребио за претрагу, већ га је само навео, на пример у питању кориснику (поглавље 9.5), па се она тумачи као мера опозива меморије у одговору, а коришћење у претрази као мера њене примене. Код епизодичке меморије прибављање није тривијално, јер се из више сачуваних епизода бира најрелевантнија, па се оно мери засебно, тачношћу прибављања епизоде (*episodic_retrieval_accuracy*).

Поред сценарија из табеле 9.2, спроведене су и две посебне провере, чији се резултати наводе одвојено: провера ажурирања меморије (поглавље 9.7) и провера поштовања процедуралног правила (поглавље 9.8).

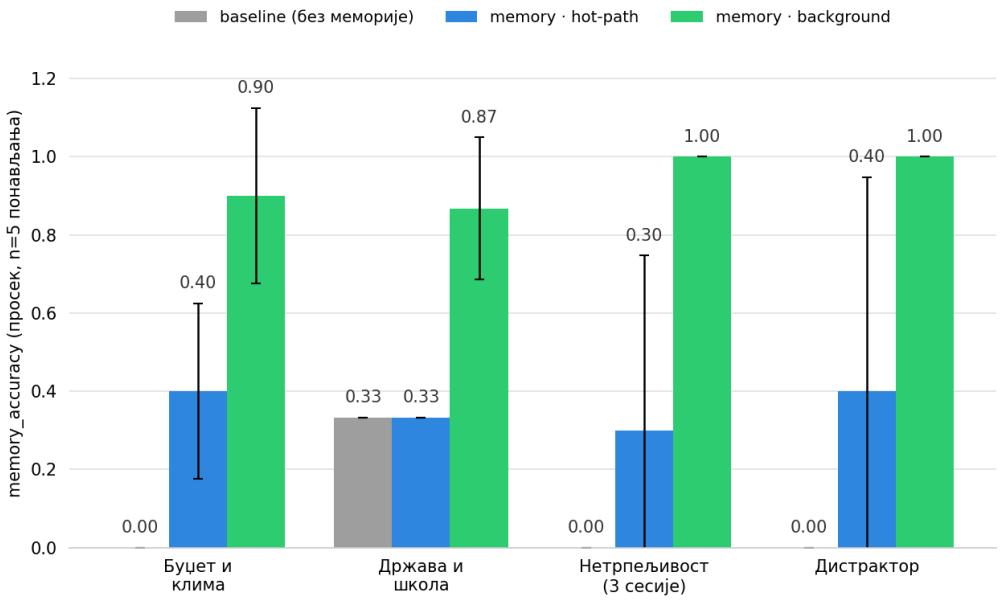
9.2. Почетни резултати: механизми екстракције меморије

Табела 9.3 и слика 9.1 приказују почетне резултате по оси механизма екстракције. Режим „*background*” у сва четири сценарија знатно је тачнији и од „*baseline*”-а и од режима „*hot-path*”: по строгој метрици 0,80 до 1,00, наспрам 0,00 до 0,40. Режим „*hot-path*” по строгој метрици бољи је од „*baseline*”-а само у сценаријима „нетрпељивост” и „дистрактор”, док је у сценаријима „буџет и клима” и „држава и школа” на нивоу „*baseline*”-а (0,00). Строга метрика даје ниже вредности од првобитне

тамо где је агент маркере „погађао” из излаза алатке, на пример 0,00 уместо 0,33 у сценарију „држава и школа”.

Сценарио	Варијанта	Тачност (првобитна метрика)	Строга тачност	Конзис-тентност	Латенција (s)	Додатно кашњење (s)
Буџет и клима	„*baseline*”	0,00 ± 0,00	0,00	0,00 ± 0,00	4,21 ± 0,54	—
	„*hot-path*”	0,40 ± 0,22	0,00	0,40 ± 0,22	8,01 ± 2,33	+3,80 ± 2,26
	„*background*”	0,90 ± 0,22	0,80	0,90 ± 0,22	12,44 ± 1,27	+8,23 ± 1,25
Држава и школа	„*baseline*”	0,33 ± 0,00	0,00	0,33 ± 0,00	4,11 ± 0,17	—
	„*hot-path*”	0,33 ± 0,00	0,00	0,33 ± 0,00	5,12 ± 0,53	+1,03 ± 0,67
	„*background*”	0,87 ± 0,18	0,80	0,87 ± 0,18	10,99 ± 1,85	+6,88 ± 1,86
Нетрпељивост (3 сесије)	„*baseline*”	0,00 ± 0,00	0,00	0,00 ± 0,00	1,09 ± 0,21	—
	„*hot-path*”	0,30 ± 0,45	0,20	0,00 ± 0,00	2,16 ± 1,38	+1,06 ± 1,38
	„*background*”	1,00 ± 0,00	1,00	0,80 ± 0,27	9,27 ± 0,59	+8,18 ± 0,62
Дистрактор	„*baseline*”	0,00 ± 0,00	0,00	0,00 ± 0,00	4,03 ± 0,50	—
	„*hot-path*”	0,40 ± 0,55	0,40	0,40 ± 0,55	5,96 ± 1,54	+1,93 ± 1,83
	„*background*”	1,00 ± 0,00	1,00	1,00 ± 0,00	11,50 ± 1,08	+7,47 ± 1,39

Табела 9.3. Почетни резултати по оси механизма екстракције (прототип, $n = 5$ по комбинацији, осим режима „*hot-path*” у сценарију „држава и школа”, где је $n = 4$)



Слика 9.1. Тачност прибављене меморије по сценарију и механизму екстракције – почетни резултати (првобитна метрика)

Добитак у тачности у почетном систему пратило је знатно веће време одговора: режим „*background*” био је спорији од „*baseline*”-а за 6,9 до 8,2 s по поруци, а режим „*hot-path*” за 1,0 до 3,8 s. Према теорији, позадинска екстракција не би требало да успорава одговор, јер се извршава ван главног тока разговора. Увидом у код утврђено је да се у прототипу на позадински упис чекало пре него што се одговор врати кориснику. Исти образац користи и основни уводни пример из документације *LangMem SDK*-а [25], док библиотека за ову намену нуди механизам *ReflectionExecutor* [27], који

позадински задатак предаје на извршавање и одмах враћа одговор.

У унапређеном систему интерфејс више не чека на упис: трајна петља догађаја (поглавље 8.2) завршава започете задатке након што је одговор приказан, а упис планира *ReflectionExecutor*. Ефекат ове измене на време одговора приказан је у поглављу 9.4.

9.3. Почетни резултати: типови меморије

Табела 9.4 приказује почетне резултате по оси типа меморије. Варијанта само са епизодичком меморијом („*episodic_only*”) ни у једном сценарију се није разликовала од „*baseline*”-а. Разлог је то што је прототип епизоде само уписивао, а никада их није учитавао назад у контекст агента (поглавље 8.8), па оне нису могле да утичу на одговор. Овај налаз био је непосредан повод за увођење прибављања епизода у унапређеном систему.

Сценарио	**„*baseline*”**	**Тип меморије**	**„*hot-path*”**	**„*background*”**
Буџет и клима	0,00	само семантичка („*semantic_only*”)	0,10	1,00
		само епизодичка („*episodic_only*”)	0,00	0,00
Држава и школа	0,33	само семантичка („*semantic_only*”)	0,33	0,93
		само епизодичка („*episodic_only*”)	0,33	0,33
Нетрпељивост (3 сесије)	0,00	само семантичка („*semantic_only*”)	0,00	1,00
		само епизодичка („*episodic_only*”)	0,00	0,00
Дистрактор	0,00	само семантичка („*semantic_only*”)	0,10	1,00
		само епизодичка („*episodic_only*”)	0,00	0,00

Табела 9.4. Почетни резултати по оси типа меморије (прототип, првобитна метрика, средња вредност за $n = 5$)

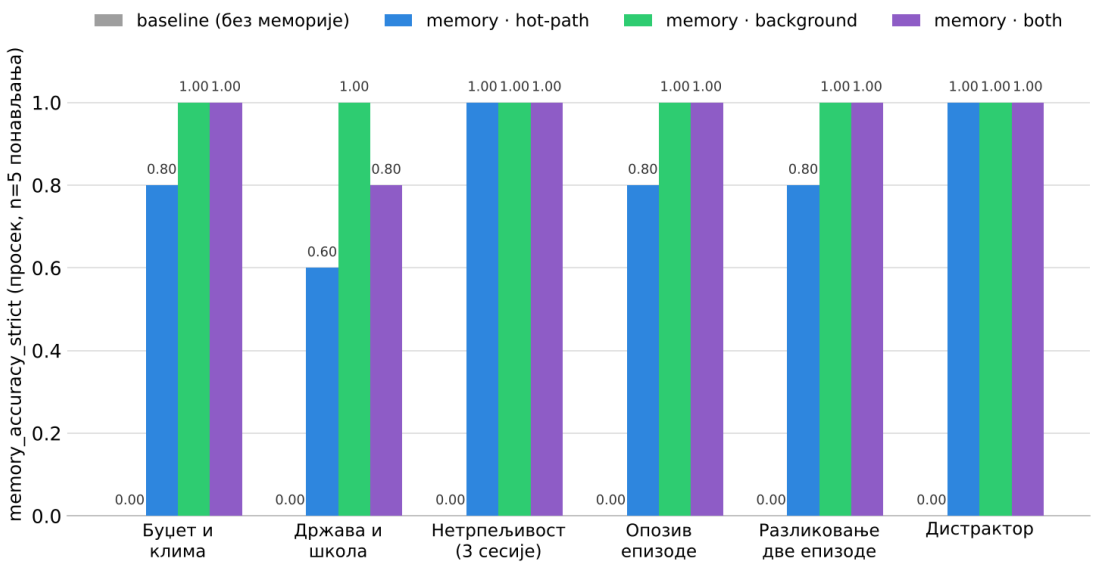
Варијанта само са семантичком меморијом („*semantic_only*”) понашала се врло различито у два режима: у режиму „*hot-path*” тачност је готово иста као код „*baseline*”-а, а у режиму „*background*” висока (0,93 до 1,00). Пошто се профил у оба режима на исти начин уграђује у *prompt*, разлика је приписана поузданости уписа – у режиму „*hot-path*” агент сам одлучује када ће позвати алатку за упис, док се позадинска екстракција покреће после сваке поруке. Каснија анализа (поглавље 9.5) показала је да на ову разлику у великој мери утиче и формулација *prompt*-а. Вредност 0,33 у сценарију „држава и школа” потиче од маркера „*school*”, који се налази у излазу алатке (поглавље 9.1), па не указује на памћење.

9.4. Завршни резултати унапређеног система

Након завршетка имплементације цела евалуација поново је извршена над унапређеним системом: шест сценарија, по пет понављања за сваку комбинацију, укупно 330 покретања (11 варијанти × 6 сценарија × 5 понављања; „*baseline*” је покренут засебно за обе осе поређења), уз посебне провере ажурирања меморије и поштовања процедуралног правила. Позадински упис планира *ReflectionExecutor*, а епизоде се прибављају претрагом по сличности („*similarity*”). Ово су резултати након измене *prompt*-а описане у поглављу 9.5; резултати исте евалуације са првобитним *prompt*-ом дати су за поређење у табели 9.8. Завршни резултати приказани су у табелама 9.5 и 9.6 и на слици 9.2. Латенција се сада мери до тренутка када корисник добије одговор, а не, као у прототипу, до завршетка уписа, па није директно упоредива са почетним резултатима. Тек након тога, пре следеће поруке, скрипта за евалуацију сачека да се позадински уписи заврше, тако да свака наредна порука и сесија раде над ажурном меморијом.

Сценарио	Варијанта	Строга тачност	Конзис-тентност	Латенција (s)	Додатно кашњење (s)
Буџет и клима	„*baseline*”	0,00 ± 0,00	0,00	5,4 ± 1,0	—
	„*hot-path*”	0,80 ± 0,45	0,80	10,3 ± 1,2	+4,9 ± 0,8
	„*background*”	1,00 ± 0,00	1,00	9,6 ± 0,5	+4,2 ± 0,9
	„*both*”	1,00 ± 0,00	1,00	10,3 ± 1,3	+4,9 ± 1,8
Држава и школа	„*baseline*”	0,00 ± 0,00	0,00	5,1 ± 0,5	—
	„*hot-path*”	0,60 ± 0,22	0,60	8,5 ± 1,7	+3,4 ± 1,9
	„*background*”	1,00 ± 0,00	1,00	9,0 ± 0,7	+4,0 ± 0,3
	„*both*”	0,80 ± 0,27	0,80	8,7 ± 2,2	+3,6 ± 2,5
Нетрпељивост (3 сесије)	„*baseline*”	0,00 ± 0,00	0,00	1,0 ± 0,1	—
	„*hot-path*”	1,00 ± 0,00	1,00	8,0 ± 0,9	+7,0 ± 0,9
	„*background*”	1,00 ± 0,00	0,80	8,7 ± 0,4	+7,7 ± 0,5
	„*both*”	1,00 ± 0,00	1,00	8,1 ± 0,5	+7,0 ± 0,5
Опозив епизоде	„*baseline*”	0,00 ± 0,00	0,00	4,7 ± 0,9	—
	„*hot-path*”	0,80 ± 0,45	0,80	5,4 ± 1,2	+0,7 ± 0,8
	„*background*”	1,00 ± 0,00	1,00	4,7 ± 0,4	0,0 ± 0,6
	„*both*”	1,00 ± 0,00	1,00	5,0 ± 0,5	+0,3 ± 0,7
Разликовање две епизоде	„*baseline*”	0,00 ± 0,00	0,00	5,5 ± 0,3	—
	„*hot-path*”	0,80 ± 0,45	0,80	6,4 ± 0,4	+0,8 ± 0,5
	„*background*”	1,00 ± 0,00	1,00	5,5 ± 0,4	−0,1 ± 0,5
	„*both*”	1,00 ± 0,00	1,00	6,2 ± 0,7	+0,7 ± 0,8
Дистрактор	„*baseline*”	0,00 ± 0,00	0,00	5,0 ± 0,5	—
	„*hot-path*”	1,00 ± 0,00	1,00	10,3 ± 0,6	+5,4 ± 0,6
	„*background*”	1,00 ± 0,00	1,00	8,0 ± 0,7	+3,0 ± 0,9
	„*both*”	1,00 ± 0,00	1,00	10,5 ± 1,1	+5,5 ± 0,8

Табела 9.5. Завршни резултати по оси механизма екстракције (унапређени систем, $n = 5$ по комбинацији)



Слика 9.2. Строга тачност по сценарију и механизму екстракције – завршни резултати

Сва три режима са меморијом знатно надмашују „*baseline*”, који је у сваком сценарију на 0,00. Режим „*background*” постиже строгу тачност 1,00 у свих шест сценарија, режим „*both*” у пет, а режим „*hot-path*” у два; све остале вредности су између 0,60 и 0,80. Ове разлике су мале у односу на варијабилност модела при $n=5$ – у сценарију са једном групом маркера већ једно неуспешно понављање од пет спушта средњу вредност за 0,20 – па се не може поуздано тврдити да је неки од режима тачнији. Изузетак је разлика 0,60 наспрам 1,00 у сценарију „држава и школа”, која је на граници статистичке значајности (пермутациони тест, $p \approx 0{,}05$). Стопа лажног памћења, која се мери у сценаријима са дистрактором и са две епизоде, износи 0,00 у свим комбинацијама осим режима „*hot-path*” у сценарију разликовања две епизоде, где износи 0,20 (лажни податак појавио се у једном од пет понављања).

Разлагање строге тачности на упис и коришћење профила (поглавље 9.1) показује где настају преостале разлике међу режимима. У четири сценарија са профилом профил је на крају прве сесије садржао све очекиване податке у свих 60 понављања (четири сценарија, три режима, по пет понављања). Разлике међу режимима настају тек при коришћењу профила: удео група маркера које је агент у новој сесији унео у упит алатке за претрагу износи у просеку 0,90 у режиму „*hot-path*”, 0,95 у режиму „*both*” и 1,00 у режиму „*background*”, док строга тачност у истим сценаријима износи 0,85, 0,95 и 1,00. Ова два показатеља се не поклапају у потпуности, јер одговор може да наведе сачувани податак и без претраге (у завршној евалуацији по један такав одговор у режимима „*hot-path*” и „*both*”, оба у сценарију „држава и школа”), али и да изостави део података са којима је претрага покренута, на пример када наведе градове из само једне од две тражене државе. Исти образац показују и варијанте само са семантичком меморијом, код којих је профил у ова четири сценарија такође уписан у свих 60 понављања. Након измене *prompt*-а упис, дакле, више није ограничавајући фактор, већ то постаје коришћење сачуване меморије.

Захваљујући неблокирајућем упису, у оба сценарија епизодичке меморије додатно кашњење је мало (до 0,8 s), а код режима „*background*” га практично нема (од −0,1 до 0,0 s). У четири сценарија са претрагом градова сва три режима са меморијом спорија су од „*baseline*”-а за 3,0 до 7,7 s, али та разлика не потиче од чекања на упис. Агент са меморијом у наредној сесији одмах изводи претрагу градова на основу запамћених преференци, док агент без меморије најчешће само тражи од корисника да их поново наведе, што је знатно краће.

Епизодичка меморија, која у почетном систему није имала ефекта, сада се прибавља и користи. У сценарију опозива епизоде агент у новој сесији тачно наводи разлог претходне претраге, а у сценарију разликовања две епизоде разлог праве од две претраге. Режим „*background*” у оба сценарија постиже строгу тачност 1,00, а најбоља прибављена епизода у свим понављањима је била она тачна (тачност прибављања епизоде 1,00). У сценарију разликовања две епизоде профил у овом режиму ни у једном понављању није садржао разлог друге претраге, па је тачан одговор потекао из прибављене епизоде. У режимима „*hot-path*” и „*both*” агент у овим сценаријима одговара углавном на основу напомена у семантичком профилу (поглавље 9.5), што објашњава вредност 0,80 код режима „*hot-path*”. Прибављање у овим режимима ни у једном понављању није вратило епизоду, јер агент у порукама у којима је уписивао профил алатку за бележење епизоде није позивао; у четири од пет понављања разлог претраге налазио се у напоменама профила, а у петом профил није био уписан. У варијанти само са семантичком меморијом у режиму „*hot-path*” агент у овим сценаријима ни у једном понављању није уписао профил, па је тачност 0,00 (табела 9.6).

Табела 9.6 приказује завршне резултате по оси типа меморије и потврђује да два типа меморије покривају различите потребе. У четири сценарија у којима је одлучујући профил корисника варијанте само са семантичком меморијом постижу од 0,60 до 1,00, а варијанте само са епизодичком меморијом 0,00. У два сценарија епизодичке меморије однос је обрнут: епизодичка меморија даје 1,00 у сва три режима, док семантичка успева само делимично, јер профил може да сачува разлог претраге у пољу напомена, али не може да разликује две претраге. Због тога се код варијанти само са семантичком меморијом у сценарију разликовања две епизоде јавља и стопа лажног памћења од 0,40 у режиму „*background*” и 0,20 у режиму „*both*”: одговор о другој претрази помиње податак из прве.

Сценарио	**„*baseline*”**	**Тип меморије**	**„*hot-path*”**	**„*background*”**	**„*both*”**
Буџет и клима	0,00	само семантичка	1,00	1,00	1,00
		само епизодичка	0,00	0,00	0,00
Држава и школа	0,00	само семантичка	0,80	0,90	0,60
		само епизодичка	0,00	0,00	0,00
Нетрпељивост (3 сесије)	0,00	само семантичка	1,00	0,80	1,00
		само епизодичка	0,00	0,00	0,00
Опозив епизоде	0,00	само семантичка	0,00	1,00	1,00
		само епизодичка	1,00	1,00	1,00
Разликовање две епизоде	0,00	само семантичка	0,00	0,20	0,80
		само епизодичка	1,00	1,00	1,00
Дистрактор	0,00	само семантичка	1,00	1,00	1,00
		само епизодичка	0,00	0,00	0,00

Табела 9.6. Завршни резултати по оси типа меморије – строга тачност (унапређени систем, $n = 5$ по комбинацији)

Три стратегије претраге епизода („*similarity*”, „*keyword*” и „*hybrid*”) упоређене су у сценарију опозива епизоде, у сва три режима и са по пет понављања. Све три су у свим понављањима прибавиле тачну епизоду (тачност прибављања 1,00). Пошто корисник у овом сценарију има само једну епизоду, резултат потврђује да све три

стратегије исправно раде, али не показује која је боља; за такво поређење потребан је сценарио са више епизода по кориснику. У завршној евалуацији коришћена је стратегија „*similarity*”.

9.5. Утицај формулације *prompt*-а на коришћење меморије

Прва потпуна евалуација унапређеног система, извршена са првобитном верзијом *prompt*-а, дала је слику сличну почетним резултатима: режим „*background*” постизао је строгу тачност од 0,80 до 1,00, а режими „*hot-path*” и „*both*” у сценаријима са семантичким профилом углавном знатно мање (табела 9.8; изузетак је режим „*both*” у сценарију „дистрактор”, са 1,00: у два од пет понављања агент ни ту није покренуо претрагу, али је у питању кориснику навео сачувани буџет и државу, што метрика бодује). Логови су показали да узрок није у упису – профил је у свим понављањима био исправно сачуван – већ у његовом коришћењу. У наредним сесијама агент у сваком од режима „*hot-path*” и „*both*” у 22 од 25 порука није позвао ниједну алатку, већ је од корисника тражио да поново наведе преференце, понекад чак наводећи сачувану преференцу као пример. У режиму „*background*” то се није догодило ниједном (0 од 25).

Да би се утврдио узрок, спроведени су мали циљани огледи у сценаријима „буџет и клима” и „држава и школа”, са по пет понављања. Профил је у њима уписан директно у складиште, без учешћа модела, а агенту је послата само прва порука наредне сесије. Тако је коришћење профила посматрано одвојено од његовог уписа. Резултати су приказани у табели 9.7.

Режим	Верзија *prompt*-а	Сценарио	Претрага покренута	Профил у упиту	Строга тачност
„*hot-path*”	првобитни	Буџет и клима	2/5	0/5	0,00
		Држава и школа	0/5	0/5	0,20
„*hot-path*”	првобитни + инструкција о коришћењу профила	Буџет и клима	5/5	1/5	0,20
		Држава и школа	5/5	5/5	1,00
„*hot-path*”	без упутстава за алатке	Буџет и клима	5/5	5/5	1,00
		Држава и школа	4/5	4/5	0,80
„*background*”	првобитни (контрола)	Буџет и клима	4/5	4/5	0,80
		Држава и школа	5/5	5/5	1,00
„*hot-path*”	коначни	Буџет и клима	5/5	5/5	1,00
		Држава и школа	3/5	3/5	0,70
„*background*”	коначни	Буџет и клима	5/5	5/5	1,00
		Држава и школа	5/5	5/5	0,70

Табела 9.7. Коришћење унапред уписаног профила у првој поруци наредне сесије, у зависности од верзије *prompt*-а ($n = 5$ по ћелији)

Огледи показују да узрок лежи у тексту упутстава за „*hot-path*” алатке. Са првобитним *prompt*-ом агент је претрагу покренуо у само 2 од 10 порука и ниједном није уградио профил у упит. Када су упутства уклоњена, а алатке остале доступне, агент се понашао готово исто као у режиму „*background*” (претрага у 9 од 10 порука),

што значи да узрок нису ни саме алатке ни тренутак уписа. Додатна инструкција о коришћењу профила вратила је претрагу, али не и поуздану уградњу буџета у упит (1 од 5). Вредност 0,70 у сценарију „држава и школа” не означава неуспех: одговор који наводи градове у само једној од две тражене државе добија 0,50. Строга тачност може бити већа од нуле и без покренуте претраге. Метрика проверава да ли се запамћени податак појављује у одговору, а не да ли је агент са њим обавио задатак, па бодује и одговор у ком агент сачувану преференцу само наведе у питању кориснику, као у примеру из завршне евалуације: *„I will search for towns in Texas and Arizona that have schools nearby, as you prefer. However, I need to know your budget range…”*. Такав одговор показује да је агент меморију прочитао и препознао као релевантну, али не и да ју је применио. Зато се строга тачност у овом раду тумачи као мера опозива меморије, док се њено коришћење у претрази прати засебно: у огледима колонама „Претрага покренута” и „Профил у упиту” табеле 9.7, а у завршној евалуацији показатељем коришћења у претрази (поглавља 9.1 и 9.4).

Упутства за алатке су, међутим, потребна да би агент профил уопште уписао: у огледима са измењеним упутствима профил је после прве сесије био сачуван у свега 1 од 10, односно 0 од 10 понављања, наспрам 10 од 10 са првобитним *prompt*-ом. Томе је допринела и прва верзија поруке компоненте *ToolErrorMiddleware*, која је агенту после грешке говорила да настави без чувања, па је та порука измењена (поглавље 8.10).

Коначно решење раздваја читање и упис профила. Правило о коришћењу профила (*SEMANTIC_READ_RULE*) заједничко је свим режимима са семантичком меморијом, а упутство за упис (*SEMANTIC_TOOL_USAGE_HINT*) одређује само када и како се профил уписује. Тиме се режими „*hot-path*” и „*background*” разликују само у начину уписа, а не и у *prompt*-у. Са коначним *prompt*-ом профил се уписује већ првом поруком у свих 10 понављања, без грешака. У наредној сесији агент у сценарију „буџет и клима” користи профил једнако као у режиму „*background*” (5 од 5), а у сценарију „држава и школа” нешто ређе (3 од 5, наспрам 5 од 5).

Табела 9.8 упоређује прву и завршну евалуацију унапређеног система, односно резултате пре и после измене *prompt*-а. Број порука у наредним сесијама у којима агент не позове ниједну алатку пао је у сваком од режима „*hot-path*” и „*both*” са 22 од 25 на 1 од 25. Строга тачност у четири сценарија са семантичким профилом порасла је код режима „*hot-path*” са 0,00–0,40 на 0,60–1,00, а код режима „*both*” са 0,00–1,00 на 0,80–1,00, док је режим „*background*” и пре измене имао високих 0,80–1,00. Тачност ажурирања профила у новој сесији порасла је са 0,50 (буџет), односно 0,60 (држава), на 1,00 (поглавље 9.7).

Сценарио	**„*hot-path*”**		**„*background*”**		**„*both*”**	
	пре	**после**	**пре**	**после**	**пре**	**после**
Буџет и клима	0,00	0,80	1,00	1,00	0,00	1,00
Држава и школа	0,40	0,60	0,80	1,00	0,00	0,80
Нетрпељивост (3 сесије)	0,40	1,00	1,00	1,00	0,00	1,00
Опозив епизоде	1,00	0,80	1,00	1,00	1,00	1,00
Разликовање две епизоде	1,00	0,80	1,00	1,00	1,00	1,00
Дистрактор	0,00	1,00	1,00	1,00	1,00	1,00

Табела 9.8. Строга тачност пре и после измене *prompt*-а (унапређени систем, $n = 5$ по комбинацији)

Измена *prompt*-а имала је и један споредни ефекат. У режимима „*hot-path*” и „*both*” са свим типовима меморије агент после претраге по правилу позива само једну од две алатке за упис. У претрагама у којима је уписао или ажурирао профил епизоду је забележио у свега 4 од 30 порука у режиму „*hot-path*”, односно 5 од 30 у режиму „*both*”, док ју је у претрагама без уписа профила бележио готово увек (23 од 24, односно 24 од 24). Епизоде се зато губе управо у порукама које доносе нове податке о кориснику: у оба сценарија епизодичке меморије, у којима свака претрага доноси и такве податке, епизода није забележена ни у једној од 15 претрага по режиму. Са првобитним *prompt*-ом однос је био обрнут – агент је епизоду бележио у 29, односно 32 од 33 претраге, а профил је уписивао или ажурирао у свега 15 од 33. Промена редоследа упутстава то није решила, већ је само заменила алатку која се прескаче. Варијанте само са епизодичком меморијом нису погођене – епизода се у њима бележи у свих 60 порука са претрагом – а упутство које изричито захтева оба позива остаје као правац даљег рада. Овај ефекат објашњава и пад строге тачности режима „*hot-path*” у сценаријима епизодичке меморије са 1,00 на 0,80 (табела 9.8): у завршној евалуацији прибављање у овом режиму није вратило ниједну епизоду.

Закључак овог поглавља је да формулација *prompt*-а утиче на коришћење меморије више него тренутак уписа. Предност режима „*background*” у првој евалуацији унапређеног система, а вероватно и у почетним резултатима, у великој мери је последица различитог текста упутстава, а не начина уписа. Након прецизирања *prompt*-а разлика међу режимима знатно се смањила (код „*hot-path*” режима строга тачност у сценаријима са профилом порасла је са 0,00–0,40 на 0,60–1,00), али није нестала: „*background*” је и даље благо испред „*hot-path*” режима (укупно 1,00 наспрам 0,83), при чему је највеће одступање у сценарију „држава и школа”.

9.6. Тестирање конзистентности понашања агента кроз више сесија

Конзистентност понашања посебно је важна у сценарију нетрпељивости кроз три сесије, у ком се преференца изнесе у једној сесији, а затим проверава у две нове, одвојене сесије. Краткорочна меморија (контролна тачка, везана за једну нит) ту не помаже, па исправно понашање може обезбедити само дугорочна меморија.

У почетним резултатима (табела 9.3) доследно понашање показао је само режим „*background*” (конзистентност 0,80, наспрам 0,00 код „*baseline*”-а и режима „*hot-path*”). У завршним резултатима (табела 9.5) конзистентност је висока у сва три режима: 1,00 у режимима „*hot-path*” и „*both*” и 0,80 у режиму „*background*”. Конзистентност се рачуна за свако понављање посебно: за сваку дискриминативну групу маркера проверава се да ли се, почев од прве нове сесије у којој се појави у одговору, појављује и у свакој наредној сесији, а резултат понављања је удео група које тај услов испуњавају. Сценарио нетрпељивости има само једну дискриминативну групу („*cold*” или „*winter*”), па је резултат сваког понављања 0 или 1, а вредност у табели је просек пет понављања. У режиму „*background*” четири понављања имају вредност 1, а једно 0, што даје 4/5 = 0,80. У том понављању агент у првој новој сесији преференцу наводи као „*no snow during winter*”, а у другој само као „*no snow*”, без речи „*cold*” или „*winter*” које метрика тражи (реч „*snow*” није дискриминативна, јер се налази у излазу алатке). Строга тачност истог понављања ипак износи 1, јер она проверава само да ли се група појавила у било којој новој сесији. Агент је, дакле, сачувану преференцу применио у обе сесије, а одступање је последица метрике.

9.7. Тестирање ажурирања меморије

Механизам ажурирања семантичке меморије („учитај – спој – сачувај”, *fetch-merge-write*) проверен је посебном провером у којој корисник прво наведе једну вредност, а затим је промени. Проверене су две врсте промене – буџета (са 200.000 на 300.000 долара) и жељене државе (из Тексаса у Вермонт) – у истој и у новој сесији. Тачност ажурирања (*memory_update_accuracy*) додељује половину бода ако одговор користи нову вредност и другу половину ако не помиње стару.

У завршној евалуацији тачност ажурирања износи 1,00 за обе врсте промене, и у истој и у новој сесији, у свих пет понављања: агент препоручује градове искључиво према новој вредности, без трага старе. Исти резултат (1,00) добијен је и у почетној провери над прототипом, на примеру промене буџета у истој сесији. Са првобитним *prompt*-ом унапређеног система тачност у новој сесији била је нижа (0,50 за буџет и 0,60 за државу), јер агент тада профил у новој сесији углавном није користио, па је половину бода добијао само зато што стару вредност није поменуо (поглавље 9.5).

9.8. Поштовање процедуралног правила

Поред семантичке и епизодичке меморије, *LangMem SDK* нуди и оптимизацију процедуралне меморије – аутоматско преиспитивање и рафинисање системског *prompt*-а на основу историје разговора. У имплементираном систему ово је искоришћено за проверу да ли оптимизовани системски *prompt* и даље поуздано намеће пословно правило дефинисано у базном *prompt*-у – конкретно, да агент у свакој препоруци града обавезно наведе податке о близини болница.

Посебним тестом (*procedure_following.json*), у ком је агенту постављен упит „*Find me warm affordable towns in Florida*.”, проверено је да ли одговор поштује ово правило. У почетној провери над прототипом мера поштовања правила (*procedure_adherence*) износила је 1,00 (један упит, $n = 1$): одговор агента за сваки предложени град наводи број болница у кругу од 5, 20 и 100 километара, уз цену, температуру и податке о школама. Исти резултат (1,00) добијен је и у завршној евалуацији унапређеног система, такође над једним упитом.

Овакав налаз показује да процедурална меморија, за разлику од семантичке, не персонализује садржај одговора према појединачном кориснику, већ обезбеђује да агент доследно примењује општа правила понашања дефинисана за целу апликацију – два механизма су, другим речима, комплементарна: семантичка меморија персонализује *шта* агент препоручује, док процедурална меморија обезбеђује *како* агент то чини, кроз све интеракције подједнако.

10. ЗАКЉУЧАК

Овај мастер рад пошао је од ограничења контекстног прозора великих језичких модела, који је у пракси знатно мање искористив од декларисаног, и потребе да се оно превазиђе екстерном, дугорочном меморијом. Теоријски део приказује архитектуре ЛЛМ агената, типове меморије (семантичку, епизодичку и процедуралну) и начине њеног управљања, са освртом на *LangGraph* и *LangMem SDK*. У практичном делу изграђена је апликација за препоруку градова погодних за пресељење, која над једним *LangGraph* агентом интегрише сва три типа меморије, три режима екстракције („*hot-path*", „*background*" и „*both*"), три стратегије претраге епизода и трајно складиште у *PostgreSQL* бази са pgvector екстензијом.

Рад је реализован у две итерације. Почетни систем делимично је потврдио основну хипотезу – у режиму „*background*" агент са меморијом давао је тачније и конзистентније одговоре од агента без меморије – али је показао и три одступања од теорије: епизоде су се само уписивале, позадинска екстракција успоравала је одговор, а апликација није била отпорна на грешке алатке за ажурирање профила. Унапређени систем отклања сва три одступања и уводи режим „*both*", детерминистичко ажурирање профила и шеснаест аутоматских тестова.

Завршни резултати (шест сценарија, по пет понављања, строга метрика) потврђују основну хипотезу: сва три режима са меморијом постижу строгу тачност и конзистентност кроз сесије од 0,60 до 1,00, наспрам 0,00 код агента без меморије. Епизодичка меморија омогућава тачан опозив претходне претраге у новој сесији и разликовање две претраге (1,00 у варијантама само са епизодичком меморијом, у сва три режима). Са оба типа меморије епизода се у тим сценаријима, међутим, прибавља само у режиму „*background*": у режимима „*hot-path*" и „*both*" агент у порукама у којима уписује профил по правилу не бележи епизоду, па тачан одговор у тим режимима потиче из поља напомена семантичког профила. Лажно памћење јавља се само у сценарију разликовања две епизоде: 0,20 у режиму „*hot-path*", односно 0,20 до 0,40 код варијанти само са семантичком меморијом.

Рад је донео и два налаза која теорија није предвидела. Прво, поједини маркери налазе се већ у излазу алатке, па је уведена строга метрика, која даје поузданију слику опозива меморије, али не мери квалитет препоруке. Друго, поуздан упис није довољан – агент сачувану меморију мора и да искористи, а то у великој мери зависи од формулације *prompt*-а. Разлагање строге тачности то потврђује: у завршној евалуацији профил је у сценаријима са профилом уписан у свих 60 понављања, а разлике међу режимима настају тек при његовом коришћењу. Предност режима „*background*" у првој евалуацији унапређеног система највећим делом је била последица текста упутстава, а не тренутка уписа. Након прецизирања *prompt*-а разлика међу режимима екстракције знатно се смањила, али није нестала: „*background*" остаје благо испред „*hot-path*" режима.

Ограничења рада су мали број понављања ($n=5$), коришћење једног модела (*gpt-4o-mini*), провера процедуралног правила над само једним упитом, то што агент у режимима „*hot-path*" и „*both*" са оба типа меморије после претраге по правилу позива само једну од две алатке за упис, па се епизоде из порука са новим подацима о кориснику не бележе и поређење стратегија претраге само у сценарију са једном епизодом. Експерименти потврђују да *LangGraph* и *LangMem SDK* пружају практичну основу за дугорочну меморију агента, а да на његово понашање формулација *prompt*-а утиче више него избор режима екстракције.

ЛИТЕРАТУРА

[1] Yehudai, A., Eden, L., Li, A., Uziel, G., Zhao, Y., Bar-Haim, R., Cohan, A., & Shmueli-Scheuer, M. (2026). *Survey on evaluation of LLM-based agents* (Version 2). arXiv. https://arxiv.org/html/2503.16416v2

[2] *LLM agents – Nextra*. Prompt Engineering Guide. (2026, February 1). https://www.promptingguide.ai/research/llm-agents

[3] What is an LLM agent and how does it work? | by kerem aydın | medium. (2024, January 9). https://medium.com/@aydinKerem/what-is-an-llm-agent-and-how-does-it-work-1d4d9e4381ca

[4] *LLM agents: The complete guide for 2026*. LLM Agents: The Complete Guide for 2026. (2026, June 14). https://www.truefoundry.com/blog/llm-agents

[5] Alammar, J., & Grootendorst, M. (2024). *Hands-on large language models: Language understanding and generation*. O'Reilly Media.

[6] Bergmann, D. (2024, November 26). *What is a context window?*. IBM. https://www.ibm.com/think/topics/context-window

[7] Understanding LLM context windows: Tokens, attention, and challenges | by Tahir | Medium. (2025, February 18). https://medium.com/@tahirbalarabe2/understanding-llm-context-windows-tokens-attention-and-challenges-c98e140f174d

[8] Paulsen, N. (2026). Context is what you need: The maximum effective context window for real world limits of LLMs. *Advances in Artificial Intelligence and Machine Learning*, *06*(01), 01–26. https://doi.org/10.54364/aaiml.2026.61268

[9] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, *12*, 157–173. https://doi.org/10.1162/tacl_a_00638

[10] Letta. (2025, July 7). *Agent Memory: How to build agents that learn and remember*. https://www.letta.com/blog/agent-memory/

[11] Schmid, P. (2025, August 4). *Memory in agents,* make LLMs remember. Philschmid. https://www.philschmid.de/memory-in-agents

[12] Chugani, V. (2025, December 30). *Beyond short-term memory: The 3 types of long-term memory AI agents need*. MachineLearningMastery.com. https://machinelearningmastery.com/beyond-short-term-memory-the-3-types-of-long-term-memory-ai-agents-need/

[13] *Long-term memory* in LLM applications. Core Concepts. (n.d.). https://langchain-ai.github.io/langmem/concepts/conceptual_guide/

[14] Singh, T. (2026, August 20). *Long-term memory for AI agents: The what, why and how*. Mem0. https://mem0.ai/blog/long-term-memory-ai-agents

[15] Yu, Y., Yao, L., Xie, Y., Tan, Q., Feng, J., Li, Y., & Wu, L. (2026). Agentic Memory: Learning Unified Long-term and short-term memory management for large language model agents. *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 21457–21483. https://doi.org/10.18653/v1/2026.acl-long.981

[16] Cleveland Clinic. (2025, October 2). *What is long-term memory?* https://my.clevelandclinic.org/health/articles/long-term-memory

[17] Chavez-Mattos, L. (2026, May 3). *What is structured memory in AI agents? how to build persistent context*. MindStudio. https://www.mindstudio.ai/blog/what-is-structured-memory-ai-agents

[18] Smetannikov, I. (2025, December 9). Design patterns for long-term memory in LLM-powered architectures. https://serokell.io/blog/design-patterns-for-long-term-memory-in-llm-powered-architectures
[19] Du, P. (2026). Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. *arXiv*. https://arxiv.org/html/2603.07670v1
[20] Winks, E. (2026, April 17). *Agent memory architectures: 5 patterns and trade-offs*. Atlan. https://atlan.com/know/agent-memory-architectures/
[21] *Langgraph Overview*. Docs by LangChain. (n.d.). https://docs.langchain.com/oss/python/langgraph/overview
[22] *Persistence*. Docs by LangChain. (n.d.-b). https://docs.langchain.com/oss/python/langgraph/persistence
[23] *Langmem*. Introduction. (n.d.). https://langchain-ai.github.io/langmem/
[24] *Hot path quickstart guide*. Hot Path Quickstart Guide. (n.d.). https://langchain-ai.github.io/langmem/hot_path_quickstart/
[25] *Background quickstart guide*. Background Quickstart Guide. (n.d.). https://langchain-ai.github.io/langmem/background_quickstart/
[26] *Graph API overview*. Docs by LangChain. (n.d.-a). https://docs.langchain.com/oss/python/langgraph/graph-api
[27] *Delayed background memory processing*. Delayed Background Memory Processing. (n.d.). https://langchain-ai.github.io/langmem/guides/delayed_processing/
[28] Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1715–1725. https://doi.org/10.18653/v1/P16-1162
[29] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*, 5998–6008. https://arxiv.org/abs/1706.03762
[30] *GPT-4o mini*. OpenAI API documentation. (n.d.). https://developers.openai.com/api/docs/models/gpt-4o-mini